



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Automating Behavior-Driven Development with Large Language Models

Exploring Test Case Generation for Fintech projects

NOEL TESFALIDET

Automating Behavior-Driven Development with Large Language Models

Exploring Test Case Generation for Fintech projects

NOEL TESFALIDET

Degree Programme in Information and Communication Technology

Date: September 11, 2025

Supervisor: Elena Troubitsyna

Examiner: Philipp Haller

School of Electrical Engineering and Computer Science

Host company: Nasdaq AB

Swedish title: Automatisering av Behavior-Driven Development med stora språkmodeller

Swedish subtitle: En undersökning om generering av testfall för Fintech

Abstract

Behavior-Driven Development is used to improve collaboration across technical and non-technical teams by having test scenarios written in natural language. The scenarios are written based on test requirements manually by a tester, which often demands specific domain knowledge and a significant amount of time. To address this challenge, this thesis aims to investigate the possibility of automatically generating the test scenarios from the test requirements written in natural language by leveraging large language models. This thesis was conducted at Nasdaq, a large financial technology company, and the test scenarios were generated for internal projects at the host company. The evaluated projects include a regulatory reporting system, a management platform, and one open-source project (Flask). A testing tool was implemented using models from Amazon Bedrock to generate behavior-driven development Gherkin-formatted test scenarios directly from requirements and source code from the projects. The source code was fetched by using Retrieval augmented generation, where the codebases had been processed and stored in a vector database.

The scenarios were evaluated with quantitative metrics and qualitatively by gathering feedback from domain experts at Nasdaq. The quantitative metrics focus on syntax correctness, execution success rates, and semantic similarity between requirements, while the feedback mostly involved rating the adequacy, readability, domain relevance, and maintainability of the tests.

The results indicate that the generated Gherkin-formatted scenarios can adequately capture the intended behavior described in the requirements while also demonstrating high Gherkin syntax correctness, with 89% of scenarios being syntactically valid in the worst-performing project and 96% at best. However, the pass rates of the test execution are still relatively low across projects and vary significantly, with the highest pass rate achieved at 59 % steps passing and only 13 % in the worst project. This thesis shows that behavior-driven test scenarios are possible to generate and can support testers, managers, and non-technical stakeholders by reducing testing efforts and improving communication across teams.

Keywords

Software Testing, Large Language Models, Test Automation, Behavior-Driven Development, financial technology

Sammanfattning

Behavior-Driven Development används för att förbättra samarbetet mellan tekniska och icke-tekniska team genom att testscenarier skrivs på naturligt språk. Scenarierna skrivs baserat på testkrav manuellt av en testare, vilket ofta kräver specifik domänkunskap och en betydande tidsåtgång. För att ta itu med denna utmaning syftar denna avhandling till att undersöka möjligheten att automatiskt generera testscenarier från testkraven skrivet i naturligt språk genom att utnyttja stora språkmodeller. Avhandlingen genomfördes på Nasdaq, ett stort finansteknikföretag, och testscenarierna genererades för interna projekt hos Nasdaq. De utvärderade projekten omfattar ett system för regelbunden rapportering, en förvaltningsplattform och ett projekt med öppen källkod (Flask). Ett testverktyg implementerades med hjälp av modeller från Amazon Bedrock för att generera beteendedriven utveckling i Gherkin-format direkt från krav och källkod från projekten. Källkoden hämtades med hjälp av Retrieval augmented generation, där kodbaserna hade bearbetats och lagrats i en vektordatabas.

Scenarierna utvärderades med kvantitativa mått och kvalitativt genom att samla in feedback från domänexperter på Nasdaq. De kvantitativa mätvärdena fokuserar på syntaxkorrekthet, exekveringsfrekvens och semantisk likhet mellan kraven, medan återkopplingen främst handlade om att bedöma testernas lämplighet, läsbarhet, domänrelevans och underhållsmässighet.

Resultaten visar att de genererade Gherkin-formaterade scenarierna på ett adekvat sätt kan fånga det avsedda beteendet som beskrivs i kraven samtidigt som de uppvisar hög Gherkin-syntaktisk korrekthet, med 89 % syntaktiskt giltiga scenarier i det sämsta projektet och 96 % i det bästa. Testresultaten är dock fortfarande relativt låga i de olika projekten och varierar avsevärt, med det högsta resultatet på 59 % godkända steg och endast 13 % i det sämsta projektet. Denna avhandling visar att beteendedrivna testscenarier är möjliga att generera och att de kan stödja testare, chefer och icke-tekniska intressenter genom att minska testarbetet och förbättra kommunikationen mellan kollegor olika team.

Nyckelord

Mjukvarutestning, stora språkmodeller, testautomatisering, beteende- driven utveckling, finansiell teknik

Acknowledgments

Special thanks to my Nasdaq supervisors, Vishal Jagadale, Alina-Maria Ciocan, and Sebastien Turgeon, for their support and guidance throughout the project. Your time and guidance on everything from technical help, finding suitable projects, and acquiring resources were truly invaluable. I am very thankful to Nasdaq for the opportunity, and I am forever grateful to everyone in the business verification team for all feedback and encouragement. I also need to thank the domain experts at Nasdaq who took out time from their busy schedules to participate in the evaluations.

On the academic side, I am very grateful to my supervisor, Elena Troubitsyna, for all the seminars, valuable discussions, and feedback during the thesis. I would also like to thank my examiner, Philipp Haller, for the feedback and for enduring my barrage of emails.

Lastly, I would like to thank my family for their unwavering support and motivation throughout this journey.

Stockholm, September 2025

Noel Tesfalidet

Contents

1	Introduction	1
1.1	Background	2
1.2	Problem	2
1.2.1	Research Questions	3
1.3	Purpose	3
1.4	Goals	3
1.5	Research Methodology	4
1.6	Delimitations	4
1.7	Structure of the thesis	5
2	Background	6
2.1	Software Testing	6
2.1.1	Overview of Software Testing Methodologies	6
2.1.2	Manual testing	6
2.1.3	Automated testing	7
2.1.4	Automated test case generation	7
2.2	Behavior-Driven Development	7
2.2.1	Overview of BDD	7
2.2.2	Tools and Frameworks for BDD	8
2.3	Large Language Models in Software Testing	9
2.3.1	Automating Test Case Generation with LLM	9
2.3.2	Prompting techniques	10
2.3.3	Prompt Templates for System and User Messages	10
2.3.3.1	System Prompt Template	11
2.3.3.2	User Prompt Template	11
2.3.4	Gherkin Format & Error Types	12
2.3.5	Retrieval-Augmented Generation	13
2.4	Related work	13

2.4.1	Software Testing with Large Language Models: Survey, Landscape, and Vision	13
2.4.2	LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing	14
2.4.3	Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation	14
2.4.4	CoverUp: Coverage-Guided LLM-Based Test Generation	14
2.4.5	Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study	15
2.4.6	CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models .	15
2.4.7	Summary	15
2.4.8	Research Gap	16
3	Methods	17
3.1	Research Process	17
3.2	Projects	19
3.3	Experimental design/Planned Measurements	21
3.3.1	Test environment/test bed/model	21
3.3.2	Used Software & Hardware	21
3.4	Evaluation framework	21
3.4.1	Quantitative metrics	21
3.4.2	Qualitative metrics	23
3.4.3	Research question mapping of metrics	24
3.5	Overall Architecture	25
4	Implementation	27
4.1	Key Components	27
4.1.1	Requirement Retrieval Connector	27
4.1.2	Initial Scenario Generation Module	27
4.1.3	Syntax Error Detection Script	27
4.1.4	Feedback Generation	29
4.1.5	Scenario Regeneration	29
4.1.6	Python Step Definition Generation Module	29
4.1.7	Context Retrieval Step (RAG)	29

4.1.8	Pseudo code for overall set-up	30
4.2	Models & Parameters	30
4.2.1	Generative LLM	31
4.2.2	Embedding Models and RAG Configuration	31
4.2.2.1	Primary Embedding Model	31
4.2.2.2	Code Preprocessing for Embedding	32
4.2.2.3	Alternative/Supporting Embedding Model	32
5	Results	33
5.1	Introduction	33
5.2	Quantitative Evaluation Results	33
5.2.1	Results for the Flask Project	34
5.2.2	Results for the DARQ Regulatory Reporting Project	36
5.2.3	Results for the NordCAM Project	37
5.3	Qualitative Evaluation Results	39
5.3.1	Feedback for the Flask Project	41
5.3.2	Feedback for the DARQ Regulatory Reporting Project	43
5.3.3	Feedback for the NordCAM Project	45
5.3.4	Key themes from Open Feedback	46
6	Discussion	49
6.1	Analysis and Interpretation	49
6.1.1	Caputuring Intended Behavior (RQ1)	49
6.1.2	Impact of project types on quality and correctness (RQ2)	50
6.1.3	Effectiveness in a Large Fintech Company (RQ3)	51
6.2	Threats to Validity	52
6.2.1	Limited model Scope	52
6.2.2	Test domains	52
6.2.3	Evaluation	53
7	Conclusions and Future work	55
7.1	Conclusions	55
7.2	Limitations	56
7.3	Future work	57
7.3.1	Fintech fine-tuning	57
7.3.2	Error handling	57
7.3.3	Tree-of-Thought	57
7.3.4	Multi-modal inputs	57
7.4	Ethical, environmental & societal impact	58

References	58
A Prompts	65
A.0.1 System prompt generate BDD test scenarios	65
A.0.2 system prompt feedback BDD test scenarios	66
A.0.3 System prompt fix scenarios	67
A.0.4 System prompt step generation	67
A.0.5 template only scenarios	68
A.0.6 template feedback	69
A.0.7 template step generation	69
A.0.8 template step generation code	69

List of Figures

3.1	Shows the workflow for a user accessing and using the implementations through a chat interface on Nasdaq genAI platform	25
5.1	Primary role distributions of the 16 participants	40
5.2	This figure visually summarizes the performance of the generated BDD test cases for Flask across the four qualitative metrics in means	41
5.3	This figure presents the average Likert scale responses of each metric for the scenarios generated for Flask	42
5.4	This figure visually summarizes the performance of the generated BDD test cases for Darq across the four qualitative metrics in means	43
5.5	This figure presents the average Likert scale responses of each metric for the scenarios generated for Darq	44
5.6	This figure visually summarizes the performance of the generated BDD test cases for Nordcam across the four qualitative metrics in means	45
5.7	This figure presents the average Likert scale responses of each metric for the scenarios generated for Darq	46

List of Tables

3.1	Overview of Project Domains, Requirement complexity, and Source Code availability	19
3.2	Mapping of Evaluation Metrics to Research Questions	24
5.1	Summary of Quantitative Results for Flask Project	34
5.2	Breakdown of Gherkin Syntax Errors for Flask Project. The count represents the number of scenarios the type of error was found.	35
5.3	Summary of Quantitative Results for DARQ Project	36
5.4	Breakdown of Gherkin Syntax Errors for DARQ Project. The count represents the number of scenarios the type of error was found in.	37
5.5	Summary of Quantitative Results for NordCAM Project	38
5.6	Breakdown of Gherkin Syntax Errors for NordCAM Project. The count represents the number of scenarios the type of error was found in.	39

List of acronyms and abbreviations

AST	Abstract Syntax Tree
ATCG	Automated test case generation
BDD	Behavior-driven development
CC	Code coverage
CICD	continuous integration and continuous delivery/continuous deployment
CoT	chain-of-thought
LLM	Large language model
LOC	Lines of Code
QA	quality assurance
RAG	Retrieval-augmented generation
SBST	Search-Based Software Testing
SDLC	Software development lifecycle
SUT	System under test
TDD	Test-driven development
UI	User interface

Chapter 1

Introduction

Today, automated testing is vital in order to develop modern software systems. Automated testing allows developers to execute test cases with little human involvement, which ensures that the system behaves as intended. As code generation tools get better, productivity will increase and will therefore create a growing need for more robust testing strategies that are improving software quality, giving faster feedback, and reducing maintenance costs.

BDD is a testing methodology that has become a helpful method for encouraging more cooperation between technical and non-technical stakeholders. BDD tests focus on testing the system's intended behavior and keeping the test in natural language, assuring everyone can understand the test suite regardless of technical capability. However, BDD test cases are tedious and time-consuming to write since both the test specification written in Gherkin syntax and the code for the test have to be created. This is where this thesis attempts to explore the possibility of LLM generated BDD test cases. This would then allow developers to focus on improving the test suite or other tasks while also allowing non-technical stakeholders to read what is tested.

Previous research in this area has primarily focused on program repair and unit test generation, while the earlier stages of testing have received less attention [1]. This work aims to answer questions regarding how well LLMs can extract behavior from requirements and if it is reflected in generated test cases, while also assessing the impact of complex code (in terms of LOC and CC) on the generated test cases. Recent studies have explored prompt engineering techniques, iterative test refinement, and code decomposition strategies for high-complexity functions, and this thesis will build upon these discoveries

to improve the quality of generated BDD test scenarios [2, 3, 4].

1.1 Background

This thesis aims to reduce the labor intensive task of writing automated test cases while also contributing to reducing the time it takes to create test specifications from the requirements. This is done through generating BDD test cases with LLMs, BDD is a software development methodology that focuses on improving communication of requirements by having tests written in natural language that describe the behavior of system functionality [5]. The tests can then act as both test specifications and test cases since part of the test is written in Gherkin syntax, which is a step-by-step instruction format in natural language that uses "Given", "When", and "Then" keywords for structure. BDD translates requirements into behavior written in this Gherkin format, where the system behavior is written in natural language, which is then mapped to functions written in a programming language that performs the test execution [6]. This thesis will focus on generating tests for the Python-based Behave framework. Requirements can be structured in many different ways. However, for Nasdaq and this thesis requirements entail a User story, acceptance criteria, and system description

When generating unit tests, the context that the LLM needs is mainly just the unit that is under test. However, BDD driven test cases might need to use different parts of a code base to test the functionality of the system. In this thesis, the LLM will use the entire codebase, including doc strings and the requirements to generate the BDD test cases. The next step is to ensure that the tests that are generated are syntactically correct and execute properly. After the initial Gherkin syntax and Python tests have been generated, an iterative test refiner, similar to what Yuan et al did, will be used to execute the tests and improve the tests [3].

1.2 Problem

BDD can be extremely beneficial for a team if used correctly. The benefits are a more understandable test suite, better collaboration regardless of technical expertise, and better documentation that is automatically updated as the system changes [7] [8]. Nevertheless, despite the availability of tools and frameworks, the process of writing BDD test cases is still manual and takes a lot of time.

This is especially burdensome in domains like fintech, where requirements are often domain-specific, constantly changing, and it is vital to meet requirements in order to meet client expectations and to comply with strict regulatory compliance. Automating BDD test generation using LLM can help ease this burden by making the process faster and less manual. LLMs are able to generate unit tests of high coverage [1], and now the question is if LLMs can reliably generate readable and domain-relevant BDD test cases that support both test automation and specification in complex, real-world systems?

1.2.1 Research Questions

- Research Question 1: How accurately is the intended behavior, as described in the requirements, represented in the generated test cases?
- Research Question 2: How does varying project types impact the quality and correctness of the generated test cases?
- Research Question 3: How effective are the generated BDD test specifications in a large fintech company in terms of readability, maintainability, and domain specificity?

1.3 Purpose

As mentioned previously, BDD can be very beneficial to cross-functional teams, and the development of these test cases is time-consuming. The purpose of this thesis is to explore the possibilities of automating this development process by leveraging LLMs and to evaluate the quality of the generated test cases. The outcome of this thesis will then be useful for Nasdaq, which will have a testing tool that automatically generates test specifications/BDD test cases from requirements and source code.

1.4 Goals

The main goal of this thesis is to develop a testing tool that automates the generation of BDD test cases directly from the requirements and the code base of the SUT. The tool will be built on the Nasdaq Gen AI platform and will serve as a standalone tool and also inspiration for future work with unit test generation.

The goal has been divided into the following sub-goals:

1. Develop a tool that generates BDD test cases from requirements and source code to reduce manual workload and improve test coverage.
2. Evaluate the LLM generated BDD tests

1.5 Research Methodology

During this thesis, a very practical challenge will hopefully be solved, and the development of BDD test cases will be partly automated. The methods used to answer these questions are the following

1. Quantitative analysis: to measure test coverage, pass/failure rate, and syntax validity of the generated test cases.
2. Qualitative evaluation: interviews with domain experts to assess the adequacy, readability, maintainability, and domain relevance of the generated tests.

The method for the development of the tool will be experimental, where different prompting strategies and LLM configurations will be explored to improve the quality of the test cases. Different strategies for gathering context for the LLM will also be tested. Some of these strategies are RAG on the code base, dividing the code into smaller chunks, adding the entire codebase to the prompt, using AST, or only using module/function names for context [9].

1.6 Delimitations

This thesis will focus on the automatic generation of BDD test cases with LLMs for Python projects. Only test cases for Python and the Behave framework will be used, and no tests in other languages will be generated and evaluated. The source code of the selected projects will be limited to 200k tokens since this is the maximum context window of the LLM available at Nasdaq. This thesis will only focus on BDD tests, and no other types of tests, such as performance or security tests, will be evaluated. The LLMs used in this project will not be fine-tuned due to budget restrictions at the host company, and the models will be accessed through Amazon Bedrock.

1.7 Structure of the thesis

Chapter 2: Covers the relevant information about the topics related to this thesis, including automated testing, LLM, and BDD.

Chapter 3: Covers the methodology used in this thesis and explains the methods used to answer the research questions.

Chapter 4: Describes the development and provides a detailed description of the testing tool developed during this thesis. In Addition to this, the implementation choices are also explained in this chapter.

Chapter 5: This chapter presents the results according to chapter 3 and discuss the validity of the data

Chapter 6: Contains a discussion of the results.

Chapter 7: The conclusions chapter will summarize the results and conclusions of this thesis and also point to directions where future work is possible.

Chapter 2

Background

2.1 Software Testing

2.1.1 Overview of Software Testing Methodologies

Software testing is an important part of the SDLC, which, as mentioned in chapter 1, ensures that systems behave as intended and meet requirements. The idea is that bugs and errors are caught early as the system continues to be developed and the system becomes more complex. The methodology has changed significantly over the years, and testing has had to evolve as software became more complex. Traditionally, there are different types of testing depending on which phase the development is in. Typical types are Unit, integration, system, and acceptance testing [10].

2.1.2 Manual testing

Manual testing is a method of testing software without using tools that automate the process of testing. Manual testers use the system based on predefined test scenarios to validate the system behavior. This method of testing is usually done by domain experts or users of the system and works best when exploration of the system needs to be made. The benefit of manual testing is that human intuition is used to find bugs and errors instead of a strict and rigid set of scenarios. However, manual testing is not cost or time-effective and does not scale well in larger projects. Since the software has scaled immensely over the years, the industry usually picks automated testing, which provides faster feedback and is more scalable, but comes with an upfront cost. Nevertheless, organizations such as Nasdaq still require some manual testing

because of strict regulatory compliance and ever-changing critical systems.

2.1.3 Automated testing

Automated testing is the opposite of manual testing, where human involvement in the test execution is very low. In this method of testing, tools and frameworks execute test scenarios automatically, typically triggered in a CI/CD pipeline. Today, it is one of the most crucial software development practices and is used for all kinds of testing types, such as unit, regression, and performance testing. The benefits are faster feedback, consistency in the projects, reducing manual efforts, and allowing for higher coverage on large projects. However, automated testing imposes several issues related to maintenance. For example, if the SUT changes, then the test cases also have to reflect the changes, and there must be an environment or infrastructure where the tests can be executed, which also needs to be maintained.

2.1.4 Automated test case generation

ATCG is the next challenge, where the code for the automated test cases is generated and executed without any human involvement. By automating this process, organizations can achieve even higher coverage and hopefully identify other bugs that are difficult to find. The advancements in artificial intelligence and LLMs have created a new path for ATCG since the models are trained on vast amounts of data. The traditional method for ATCG is SBST, and LLM has also been incorporated with this method in the project CODAMOSA, where Lemieux et al. used the models to escape plateaus in coverage [11]. The method of using LLM to perform ATCG has also been used in industrial settings. For example, Meta developed a testing tool called TestGen-LLM, which uses LLMs to improve old human written test cases, resulting in a 25% increase in coverage [12]. However, there are significant challenges with using LLMs, such as quality, maintainability, flaky tests, and domain-specific language.

2.2 Behavior-Driven Development

2.2.1 Overview of BDD

BDD was invented by Dan North to capture system behavior in test cases that are accessible for anyone [6]. BDD is a software development methodology

that is similar to TDD, where the main difference is that test cases are described in natural language, reducing the gap between technical and non-technical testers. The test cases are separated into two sections, Gherkin syntax and step functions. The test cases are described in steps in any language and include some special keywords that together create the Gherkin syntax. The test cases are structured such that the behavior of the SUT and the intended results of the SUT are described. The most common method of writing the Gherkin test cases is using the Given, When, Then structure. The Given keyword lets the tester define all preconditions that are needed for the test, while the When keyword describes the actual action that has to be taken, and the Then keyword describes the final outcome. This structure makes the test scenarios clear and concise and forces the tester to write test cases based on system behavior. An example of the scenario using this syntax can be seen below 2.2.1. The other section of the test cases is the step function. The step functions are the part that executes the test scenarios. Each function corresponds to a step from the Gherkin scenario definitions. The step function can be written in different programming languages depending on the framework that is being used.

```
Feature: Money Transfer
```

```
Scenario: Successful transfer
  Given a user has $100 in account A
  When the user transfers $50 from account A
  Then account A should have $50
```

The idea of having this separation is to have the test specification/documentation for the test cases in the same place as the test execution. This allows developers and the stakeholders to collaborate in a more direct manner and hopefully improves the collaboration. In addition to this, the benefit is that the documentation changes with the test cases, which is usually a problem with other testing methods. There is also an opportunity to catch miscommunication in requirements early on, before development, since the stakeholder does not need to rely on communication from developers and vice versa.

2.2.2 Tools and Frameworks for BDD

There exists a multitude of frameworks that can execute BDD test cases for different languages. One of the most popular BDD frameworks is Cucumber

and was initially created for Ruby but now supports most common languages such as JavaScript, Python, Java [13]. The Cucumber framework uses text files (.feature files) to store the Gherkin test scenarios. The steps in the scenarios are then mapped to a step function written in the chosen programming language. Another framework is Behave, which works very similarly to Cucumber but only works for Python. Behave also uses the .feature files to store the scenarios and map the step functions by using decorators. At the moment, it is not as popular as Cucumber, but the community has been growing. SpecFlow is another framework that enables BDD testing for .NET, allowing scenarios to be written in Gherkin and step functions in C#. One type of testing that works well with BDD is UI testing [14]. In order to test UI components, tools such as Playwright or Selenium are often used to interact with web applications.

Listing 2.1: Behave test steps for account withdrawal example scenario

```
from behave import given , when , then

@given(a user has {amount:d} in account A)
def step_given_account_balance(context , amount):
    context.balance = amount

@when(the user withdraws {withdrawn:d} from account A)
def step_when_withdraw(context , withdrawn):
    context.balance -= withdrawn

@then(account A should have {remaining:d} left)
def step_then_check_balance(context , remaining):
    assert context.balance == remaining
```

2.3 Large Language Models in Software Testing

2.3.1 Automating Test Case Generation with LLM

As mentioned previously, transformer-based models have opened new opportunities within software testing, especially automation of test generation. Earlier methods of generating tests needed requirements to be written in a specific structure in order for tests to be generated [15]. However, due to

the advancements of LLM, requirements can be directly understood by the models. This thesis will leverage this and directly generate the BDD style test cases from the requirements. Some studies have been made on this topic, for example, Karpurapu et al. tested generating BDD acceptance tests directly from requirements [2]. The study showed that LLMs can directly understand requirements and generate reasonable test cases.

The majority of research in this area has been on ATCG of unit tests since LLMs can overcome limitations such as generating high coverage test suites [1]. Another popular area is the generation of test data or test inputs. For example, this data could be JSON files or even combinations of user input. Since LLMs are trained on vast amounts of data, it is possible that the models can predict user input combinations that even human testers might miss.

2.3.2 Prompting techniques

There are several methods to improve results from LLMs, one of these methods is to engineer the prompts that the LLM receives. Different prompting techniques work better for some tasks and allow the models to solve more complex tasks and make the responses more reliable. In order to generate accurate code and test cases, developers have to optimize prompts for better results and cannot rely on simply using the model directly to get good tests [16]. The expert prompting technique has shown promising results and allows the model to take on an expert role before getting the query, which has shown to improve the output [17]. The most common prompting method is Zero-shot, which is simply when the user does not include any examples of the task. This works because the LLMs is often instruction tuned from the start. Few-shot prompting simply includes examples of the task in the prompt in order to lead the response in the direction the user wants. Nevertheless, examples in the prompt won't invoke reasoning for all tasks, and this is where CoT works better. CoT have been shown to be able to evoke reasoning in LLMs by including sub reasoning steps in the prompts [18].

2.3.3 Prompt Templates for System and User Messages

To improve consistency, accuracy and user experience, structured prompt templates that separate the roles of the `system` and `user` messages are used.

2.3.3.1 System Prompt Template

The system prompt sets the model's role and comes before the instructions given in the user prompt, where the task is usually given. This prompt remains mostly constant across each user invocation and provides context for the expected output. The following is an example of the system prompt used in this thesis:

```
"You are a senior software tester skilled
in Behavior-Driven Development (BDD) .
Your task is to write well-structured BDD scenarios
in Gherkin syntax based on user requirements.
Format your output using the Given-When-Then style.
```

```
Do not include explanations-output only the scenarios"
```

This prompt ensures that the model behaves as a domain-specific assistant with the correct tone and output format. It also discourages verbose completions, which in this use case can break the test generation pipeline since the scenarios are directly used to generate Python step code. It is paramount that each model invocation only returns the expected output for the entire generation pipeline to work. The prompts used in this thesis are, therefore, much more comprehensive compared to this example and can be seen in the Appendix [A](#).

2.3.3.2 User Prompt Template

User prompts are filled with specific requirements for each test case generation task. An example of a zero-shot user prompt follows this structure:

```
Requirement: A user should be able to log in
with valid credentials.
```

In few-shot settings, examples are embedded before the new requirement such as in the following:

```
"Example:
Requirement: Users should be able to reset their
password if they forget it.
**Example output**
Feature: Password Reset
Scenario: Successful password reset
Given the user is on the "Forgot Password" page
```

```
When the user submits their registered email
Then the system should send a password reset link to the email
```

```
Now write a new scenario for the following requirement:
Requirement: A user should be able to log in with valid credentials."
```

The examples in the prompt help the model to better understand the format and scope of the desired response. This thesis uses few-shot prompting where the examples are embedded in the system instructions (see Appendix A).

2.3.4 Gherkin Format & Error Types

As mentioned previously, Gherkin is a structured and readable syntax used in ".feature" files. There are several structural rules that need to be followed in order for most BDD frameworks to map steps correctly to the executable code. Each file must start with a "Feature:" line that allows the tester to write documentation on the feature being tested, and this line can only appear once in a file. After the feature lines, the file is followed by "Scenario:" or "Scenario Outline:" blocks that define steps for each individual test case. The scenarios need to use a sequence of keywords (Given, when, then) in a set order, which can be seen in example 2.2.1.

Key formatting rules

- One Feature: Every file must start with exactly one Feature: [19]
- Scenario blocks: Each feature contains one or more scenarios (Scenario: or Scenario Outline:) with corresponding steps that follow directly after the Scenario declaration.
- Step keywords: Steps must be sequenced in the correct order
- Examples tables: A "Scenario Outline:" (template) must be followed by an Examples: table (A standalone "example:" outside a Scenario Outline block has no context and is also invalid.)

The error types (described in 3) above were chosen because they correspond to fundamental Gherkin rules, and each one stops the generated feature file from being parsed and used by the BDD frameworks. For example, missing scenarios keywords or missing "Scenario:" declaration will render the test case unusable and cause problems when the LLMs is generating the executable

code for the steps. By using these error types, all structural problems that will make a scenario unusable are caught early and could potentially be regenerated before further generation phases are started. Only structurally valid Gherkin can be executed as tests, fixing all the error types is the first step towards usable, automated scenario generation.

2.3.5 Retrieval-Augmented Generation

To generate system-level tests, the relevant context of the system code base must be used correctly in the prompts that generate the tests. However, the code bases at Nasdaq can significantly exceed the context window token length that most LLMs can process. RAG can solve this limitation by only retrieving the relevant context for generating the test. The retrieval of code is done by using semantic search on a vector database that stores the embedded code base. The embeddings are created by using an embedding model such as the *amazon.titan-embed-text-v2:0*. This semantic search query result is then used to augment the prompt in the LLM call by including the correct code context.

2.4 Related work

This section will go through some of the literature and papers that have made contributions towards software testing, ATCG, BDD, and LLM.

2.4.1 Software Testing with Large Language Models: Survey, Landscape, and Vision

Wang et al. presents an comprehensive survey with over 100 studies of the most recent research regarding software testing with LLMs [1]. The paper maps the entire research landscape and highlights the opportunities in this field and the challenges with integrating LLMs in the testing process. The authors divide the analyzed studies into several steps of the testing process and show that the majority focus on generating tests (unit tests) or bug fixing/program repair. The paper identifies some of the more challenging aspects, which are still how to generate high coverage tests and create reasonable assertions/oracles. This paper also brings up how LLMs is good at going from natural language to test scenarios, which is helpful for this thesis since the Gherkin syntax is stating the test scenario in BDD testing. The landscape study also points to some directions for future research, including

improved prompt strategies, usage of RAG, fine-tuning, and integrating code analysis tools in the test generation pipeline.

2.4.2 LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing

Xue et al. developed the testing tool LLM4Fin, which could automate the generation of test cases and achieve high coverage directly from natural language and business rules for the financial sector [20]. The study found that by fine-tuning the LLM and using algorithms combined with the model, LLM4Fin managed to show significant improvements to the results. This study indicates that fine-tuning is a variable maybe even be necessary when dealing with a very specialized domain such as fintech.

2.4.3 Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation

Karpurapu et al. evaluated LLMs ability to generate the BDD acceptance test scenarios and assessed several models, including GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2 [21]. The assessment focused on the accuracy and syntactical errors found in the generated Gherkin scenarios. GPT-3.5 and GPT-4 performed the best in generating error-free BDD scenarios when used with few-shot prompting. The paper also mentions challenges such as prompt sensitivity, domain specificity, and the need for human validation. Since the tests are generated from the requirements, they can drastically change the outcome. The study also indicates that few-shot prompting reduces most of the syntax errors in the generated test cases compared to zero-shot.

2.4.4 CoverUp: Coverage-Guided LLM-Based Test Generation

One of the challenges with generating unit test cases is to achieve high coverage. Pizzorno et al. used an iterative approach to guide the models in attaining higher code coverage. This is done by gathering and applying feedback on the generated test cases and simply trying to regenerate better

tests. This feedback is based on code coverage, static analysis, and the code context [9]. This method resulted in higher line and branch coverage. This thesis will also use this method to correct syntax errors and improve coverage.

2.4.5 Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study

Arora et al. implemented RAGTAG, which uses RAG to automatically generate test scenarios from requirements [22]. This method of using RAG managed to gather domain-specific context in a more efficient way than what is needed by the LLM to understand the requirements. The outcome of this study was test scenarios that adequately aligned with the requirements.

2.4.6 CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models

Lemieux et al. introduced CodaMOSA, an algorithm that combines Search-Based Software Testing (SBST) with LLMs from OpenAI. By using LLM-generated test cases in the SBST pipeline, CodaMOSA solves and escapes code coverage plateaus, improving the discovery of under-covered code regions [11].

2.4.7 Summary

These studies have been especially helpful for the implementation and guidance on what kinds of results can be expected. The survey by Wang et al. provides a detailed and comprehensive landscape of LLMs in software testing and shows open challenges, which this thesis addresses by focusing on BDD testing which is not as explored as other areas [1]. LLM4Fin study showed the effectiveness of domain-specific fine-tuning in a fintech context, which sets expectations on the out-of-the-box performance of the models regarding finance-specific terminology. The work by Karpurapu et al. validated the possibility of generating BDD tests using LLMs and identified prompt strategies and syntax accuracy metrics. CoverUp introduces an iterative feedback loop for improving test coverage, which this thesis uses during test refinement stages.

2.4.8 Research Gap

Automated test case generation leveraging LLMs is a relatively explored topic. However, as highlighted in the related works section, most of the existing research on the area focuses on unit tests, test inputs, and repair rather than high level acceptance tests and test planning [1]. Moreover, the context of the research is not often held in an industrial setting, especially in a fintech environment. Even fewer studies explore leveraging LLMs for BDD testing with test scenarios written in Gherkin language. Karpurapu et al. showed that large language models can produce Gherkin acceptance scenarios, but their evaluation did not consider the requirement coverage of the generated tests and used data sets with relatively simple user stories [21]. Additionally, their study noted that human evaluation is needed and the requirements used in this study was not domain-specific, which can have an impact on the quality of the test cases. Xue et al. developed the LLM4Fin model, that used a fine tuned LLM on finance domain data in order to better generate test cases [20]. However, this study did not generate tests using the Gherkin language and relied on training models to improve results. In conclusion, no prior study has investigated end to end BDD scenario generation, by using out of the box LLMs in an industrial FinTech environment, leaving a niche that has yet to be fully explored.

Chapter 3

Methods

This chapter describes the methodology for evaluating the effectiveness and quality of the generated BDD test cases. The goal of this chapter is to assess how well LLMs and the implemented testing tool can produce high quality Gherkin BDD scenarios for Python projects, and to evaluate those scenarios both quantitatively and qualitatively using expert judgment. This chapter includes information on the selection of codebases and test generation methods, followed by the evaluation methods. The methodology is structured to address the research questions by comparing different codebases and requirements.

3.1 Research Process

Step 1 Project Selection and Scenario Preparation The first step involved finding different projects and scenarios that will serve as evaluation targets. These are explained in section 3.2. For each project, a set of testable behaviors or workflows was identified or created. In the case of Flask, where explicit requirements were not available, artificial requirements were created to simulate realistic application behavior. For DARQ, existing regulatory scenarios formed the basis of the test cases, and for NordCAM, structured UI flows were selected that could be described in Given-When-Then format. This step supports RQ2, which involves answering how the quality of the tests is impacted by the projects. By selecting different projects, the generated tests can be compared across domains.

Step 2 Prompt Engineering and Test Case Generation Once the test scenarios were identified, prompts were crafted to generate BDD test

cases using a large language model accessed via Amazon Bedrock and used in the Nasdaq AI platform. The test cases were generated using different prompting strategies. Zero-shot involved presenting the model with only the requirement. Few-shot prompting included one or more example test cases to guide on what is expected. Context-augmented prompting included additional context, such as source code snippets, CSV content, static analysis, and runtime errors. The LLM was instructed to produce output in Gherkin syntax, which was then stored as .feature files for further evaluation. This step is connected to RQ1 and RQ2 since the goal is to leverage prompt techniques to generate as many correct scenarios as possible while also correctly capturing the intended behavior from the requirements.

Step 3 Implementing Testing Tool The third step focused on implementing the tool in NASDAQ's AI platform. This stage will also involve acquiring the necessary resources to initiate development, including Bedrock API access, AWS Lambda, GitLab runners, and a vector database. Across all SUT, the tests were checked for syntax validity before being returned to the user, with potential unfixed errors, and the outcomes are logged.

Step 4 Evaluation After test execution, the generated test scenarios are distributed to the domain experts for each project in the form of a survey for evaluation and review. In the meantime, the tool will generate test cases for the same projects and requirements multiple times to collect quantitative data. The quantitative metrics included syntax correctness (whether the scenario followed proper Gherkin and Python syntax), error types, execution pass rate, number of scenarios, and LOC generated [2]. The qualitative evaluation was conducted through a structured survey with 16 domain experts. Each participant reviewed a selection of generated BDD scenarios and rated them based on Adequacy (meaning how well the requirements are covered), readability, maintainability, and domain relevance. The reviewers were also invited to provide open feedback for each answer, which helped surface qualitative insights that may not be captured through numeric metrics alone. This feedback was also used by Nasdaq to evaluate the usability of the generated scenarios for other software projects. Since evaluating test cases is somewhat subjective, it is paramount to have both qualitative data together with quantitative data to answer the research questions stated in the introduction chapter.

3.2 Projects

Table 3.1: Overview of Project Domains, Requirement complexity, and Source Code availability

Project	Domain	Complexity	Source Code
Flask	Open-source web app framework	Low	Yes
DARQ	Regulatory	Medium	No
NordCam	Web UI Testing	High	Yes

Each project’s requirements have different complexity levels. Complexity is determined by the number and detail of scenarios requested in the requirements. As shown in Table 3.1, Flask has the lowest complexity, DARQ is medium, and Nordcam has the highest. Simple requirements with few general behaviors are deemed as low complexity, while requirements with many specific rules, workflows, or scenarios are high complexity. Flask is low complexity since the requirements used were only one broad scenario, which covers the Flask server startup capabilities. The DARQ requirements are medium complexity since they contain six different scenarios that are connected to different compliance rules. These scenarios are moderately detailed and require specific assertions (such as data validations or threshold conditions) but do not involve complex user interactions. NordCAM has the most complex requirements that involve detailed sequences of front-end actions such as navigating menus, entering fields, and clicking buttons. Moreover, 12 different scenarios are required in the requirements, meaning that 12 different elements on the front end need to be found and tested. The scenarios in the requirements are both numerous and highly specific, therefore, the Nordcam project requirements have the highest complexity.

Each project was picked to appropriately cover the different dimensions for BDD testing and use cases for Nasdaq. Flask is a simple baseline with a general requirement scenario. It is an open-source project that allows testing the basic generation capability of the LLMs. DARQ is a business-rule-oriented project without source code and evaluates the use cases regarding compliance testing. NordCAM is the most complex use case with UI based workflows. this project evaluates the model’s performance on frontend, UI test scenarios, which are highly prevalent in web application testing.

- **Flask** is a lightweight, popular Python web framework made for building web applications and APIs. This project fits this thesis since its source code is under 200k tokens and its behaviors map naturally to BDD style scenarios. For example, a feature such as “submitting a contact form” or “logging into an account” can be described and tested using Given-When-Then steps. However, it isn’t easy to find well-defined requirements, so requirements will be created to test its routing capabilities.
- **DARQ Regulatory Reporting** Besides open source Python projects, this study includes a project from Nasdaq. DARQ is used to secure accuracy according to MiFID II regulations on the reports submitted to the legal authorities. DARQ does not have source code and performs the test scenarios on reports in CSV format, which are generated via Nasdaq’s internal reporting tool. These reports are validated for compliance using semi-structured business rules, and the idea is that the implementation will take the business rules as requirements and create BDD Gherkin syntax and step definitions that perform the testing according to business rules. This project will give insight into how the generated tests perform without any source code and the ability to cover the requirements.
- **NordCAM** is Nasdaq’s change management system for the trading systems, and it provides a user interface for configuring various market related entities such as instruments, indices, issuers, and participants. This system enables the configuration of critical trading infrastructure, including trading calendars, list populations, and the suspension or halting of participants and order books. The web application is written in multiple languages and is a relatively small code base.

NordCAM is selected as a project because of its UI and well defined user workflows, which make great BDD scenarios. Unlike DARQ, NordCAM allows for the definition of behavioral flows through front-end actions. The test generation will focus on user interaction scenarios, such as suspending a clearing participant. For example, one feature is the “Clearing Suspension Automation,” which involves step-by-step UI interactions that can be directly translated into Given-When-Then scenarios. This project provides insight into the performance of LLMs generated test cases in UI scenarios where the SUT is accessed only through UI and not programmatically.

3.3 Experimental design and Planned Measurements

3.3.1 Test environment/test bed/model

The Nordcam and Darq project at Nasdaq is not open to the public except for the requirements. However, it is possible to reproduce the experiment using the Python library Flask by recreating the implementation as described in Chapter 4.

3.3.2 Used Software & Hardware

- Windows 11
- Python 3.12.8
- Gherkin parser [23]
- Behave 1.2.6 [24]
- AWS Bedrock
- AWS Lambda
- PostgreSQL + PGVector
- Claude 3.5 Sonnet

3.4 Evaluation framework

The evaluation of the generated test cases was conducted in two separate parts. The quantitative part measured outcomes from test execution and generation, and the qualitative part used expert judgment on the usefulness of the test cases. This section describes the framework used to evaluate the results, including the specific metrics.

3.4.1 Quantitative metrics

- Syntax Correctness: Output test files were syntactically correct in both Gherkin. The type of error and its count are included in this metric. For example, if one of the scenarios is missing a "Given" keyword, or if a step definition is missing.

- **Pass Rate:** the number of generated steps that passed during execution on average. The pass rate is computed per project, meaning the number of steps out of the total generated for that project that passed directly.
- **Fail rate:** Similarly to the pass rate, the fail rate is the number of generated steps that fail during execution.
- **Skip rate:** If a step fails, then the following steps in the scenario are skipped and can not be executed. Therefore, the skip rate is the number of steps that are skipped after a step failure on average.
- **Undefined rate:** If the Behave framework can't find a step function for a step, then the step is counted as undefined. Therefore, the skip rate is the number of steps on average that are not generated.
- **Scenario Count:** The total number of individual scenarios generated within a feature file or set of files for a given requirement set and SUT. This indicates the volume of tests generated.
- **Avg Step Count:** The average number of steps (Given, When, Then, And, But) per scenario. This illustrates the granularity and length of the generated scenarios. Also, a high step count means more code, indicating that the scenarios are more complex to maintain.
- **Ambiguity Rate:** A measure indicating potential ambiguity in step definitions, by identifying steps that contain vague and imprecise wording, and measures the percentage of ambiguous steps generated. Higher ambiguity rate indicates low readability and makes it harder to use the scenarios as documentation. A step is marked as ambiguous if it contains a word from the following list: ["it", "thing", "stuff", "some", "this", "any", "that", "etc", "many", "something", "info", "several", "value"].
- **Reused Steps:** The number of reused step definitions found across all generated scenario steps for a given generation run. A higher number of reused steps suggests the LLM generated steps that map efficiently to reusable code, indicating better maintainability of the test code.
- **Semantic similarity:** speaks to the likeness of the scenario to the requirements. Low cosine similarity indicates that the scenarios are very different from the requirements, which speaks to the scenarios' ability to be used as documentation and whether the scenarios cover

the requirements. Really high scores would mean no difference, which means that the test generation might have failed to branch out and find functional test scenarios.

3.4.2 Qualitative metrics

This thesis uses a survey based evaluation with domain experts to evaluate the quality of the generated BDD scenarios. Experts rate the following metrics on a 5-point Likert scale from e.g., 1 “Very hard to understand” to 5 “Very clear”, and/or provide comments

- **Adequacy (Requirement Coverage):** metric to ensure that the generated test scenarios adequately cover the requirement or user story. This criterion checks if the LLM correctly understood and translated the requirement into test steps, or if it missed the point.
- **Readability:** BDD is intended to be readable by non-developers as well, so the generated Given-When-Then steps need to be phrased in natural, understandable language.
- **Maintainability:** Does the scenario appear maintainable in a real project, and are the scenarios concise, since too many unnecessary steps only cause extra maintenance.
- **Domain Relevance:** This metric checks if the scenarios have captured project-specific language and use domain terminology.

3.4.3 Research question mapping of metrics

Research Question	Quantitative Metrics	Qualitative Evaluation
RQ1: Accuracy of intended behavior <i>How accurately is the intended behavior, as described in the requirements, represented in the generated test cases?</i>	Semantic Similarity between each scenario and its requirement (measures alignment with requirement content)	Adequacy: Experts judge how well each test scenario covers the requirement
RQ2 – Impact of project type on quality and correctness <i>How does varying project types impact the quality and correctness of the generated test cases?</i>	- Syntax Correctness per project (percentage of scenarios with no Gherkin syntax errors) - Execution Outcomes per project (e.g., Pass Rate, Fail/Skip/Undefined rates) - Scenario and Step Counts (total number of scenarios and average steps per scenario)	- Open Feedback - Domain Relevance Rating: Use of correct domain-specific terminology
RQ3 – Effectiveness of BDD specifications <i>How effective are the generated BDD test specifications in a large fintech company in terms of readability, maintainability, and domain specificity?</i>	- Ambiguity Rate in steps (frequency of vague or unclear steps, lower ambiguity means more clear and readable scenarios) - Reused Steps (how many step definitions are reused; more reuse suggests better maintainability) - Average Steps per Scenario (shorter, focused scenarios are easier to maintain)	- Readability Rating: Clarity to non-technical stakeholders - Maintainability Rating: Ease of updating and managing scenarios - Domain Relevance Rating: Use of correct domain-specific terminology

Table 3.2: Mapping of Evaluation Metrics to Research Questions

3.5 Overall Architecture

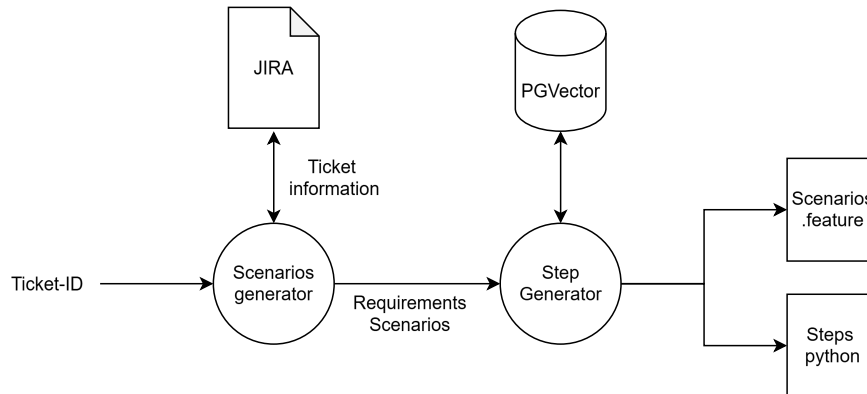


Figure 3.1: Shows the workflow for a user accessing and using the implementations through a chat interface on Nasdaq genAI platform

The testing tool is deployed as a service on the Nasdaq AI platform by containerizing the project and using AWS Lambda for execution. The the system is designed to take user input, automatically retrieve relevant requirements, use LLMs to generate and improve the tests, and Output the generated BDD test cases. The pipeline entails several steps for Gherkin scenario generation with an iterative refinement loop and Python step definition generation uses different LLM calls and context retrieval mechanisms.

The process is initiated by the user providing a Jira ticket ID. This input serves as the starting point, and the tool fetches the user story, description, acceptance criteria, and the project name (used for RAG metadata filter). The prompts are built from this information in the later generation steps.

The prompt templates used in the implementation are short and mainly contain the required context and the task at hand for the module. This was done to allow the context (requirements, code, etc) to take as many tokens of the context window as possible. The majority of task information, such as the detailed instructions, persona assignment, formatting rules, and constraints, was moved into the System Prompts. The user prompts and system prompt templates can be found in the appendix [A](#)

Chapter 4

Implementation

This chapter describes the design and implementation details of the automated BDD test case generation tool. The tool was developed to automate the creation of Gherkin scenarios and corresponding Python step definitions by using LLMs and integrating the tool with existing platforms at Nasdaq.

4.1 Key Components

The tool is composed of several modules that are responsible for specific tasks within the test generation pipeline.

4.1.1 Requirement Retrieval Connector

Based on the provided Jira ticket ID, this module automatically fetches the user story or requirements associated with the ticket directly from the Nasdaq Jira instance. The connector uses the Nasdaq Jira base URL and adds on the required certificates to access the tickets with the Jira Python library [25]

4.1.2 Initial Scenario Generation Module

Using the retrieved requirements, this module initiates the generation of Gherkin scenarios (.feature files). The user prompt is created with the requirements and a few examples for the LLM to make the initial test suite.

4.1.3 Syntax Error Detection Script

The error detection script is important to the iterative refinement loop since it was made to find errors in the generated scenarios so the LLM can know what

to fix in the coming iterations. The script reads the generated `.feature` file line by line and keeps track of the state based on previously seen keywords to find lines that go against the Gherkin syntax rules.

It iterates through each line, ignoring comments and empty lines, and uses helper functions (usually to tell if a line is a type of step) and state variables such as `in_scenario` to track the current context within the feature file.

The script finds and counts occurrences of several specific syntax and structural errors, including:

- **Multiple Features:** The start of each `.feature` file should begin with only one `Feature:` line. The script returns any other occurrences.
- **Missing Feature:** Verifies that the mandatory `Feature:` declaration is present in the file.
- **Missing Scenario:** Checks if at least one `Scenario:` or `Scenario Outline:` is there, since a feature should describe at least one behavior.
- **Background Inside Scenario:** Background sections, which define preconditions (similar to setup), must be placed before any `Scenario` or `Scenario Outline`. This error is counted if a `Background:` is found after a scenario has started.
- **Step Outside Scenario:** Gherkin keywords (`Given`, `When`, `Then`, `And`) must be placed within a `Scenario`. Steps found outside a scenario are returned as an error.
- **Given After When/Then:** Maintains that the order of `Given` steps are placed before `When` steps and `Then` steps within a scenario.
- **When After Then:** Maintains that the order of `When` steps come before `Then` steps.
- **Examples Outside Scenario Outline:** The script also captures `Examples:` tables that appear without being in the part of a scenario block.

The script returns a dictionary where the keys are the error types and the values are the number of times the error appeared in the file. This output is then passed to the Feedback Generation Module, providing the context to the LLM to understand the specific problem that needs to be corrected and guide the scenario generation towards more syntactically correct scenarios.

4.1.4 Feedback Generation

This component utilizes another call to a Claude 3.5 model. It is provided with the original requirements and the error output from the detection script to generate readable feedback and correction instructions for the scenario generation LLM. This feedback can also be focused on specific types of testing, such as negative and/or positive testing, if the user adds this to the Jira ticket. The output of this module will be given to the Scenario Regeneration module.

4.1.5 Scenario Regeneration

The previously generated scenarios are again sent to a Claude 3.5 model in this module with the original requirements, the last error output, and the generated feedback. This process iterates for a maximum of 5 times or until the error detection script reports no remaining Gherkin syntax errors. The maximum number of iterations can be increased, but uses 5 iterations as the default value. If this limit is hit during executions, the test suite with the lowest amount of errors returned from the script is given to the user.

4.1.6 Python Step Definition Generation Module

Once the scenarios are done (or after the maximum iterations), this module generates the corresponding Python step definitions needed by the Behave framework. A prompt is sent to a Claude 3.5 model that includes the requirements and relevant code context to create the Python functions that implement the Gherkin steps.

4.1.7 Context Retrieval Step (RAG)

To aid the step definition module in generating accurate Python step definitions that interact correctly with the project's code, a context retrieval mechanism is used. This module uses Langchain to interface with a PGVector database containing vector embeddings of the project's codebase. The requirements are used as a query to retrieve relevant code snippets, which are then included in the prompt for the step definition generation.

4.1.8 Pseudo code for overall set-up

Algorithm 1 LLM-based BDD Test Generation Pipeline

Require: Jira ticket ID

Ensure: Gherkin feature file and Python step definitions

```

1: // Step 1: Fetch requirements from Jira
2: reqs  $\leftarrow$  get_requirements(jira_id) {Returns description,
   user story, acceptance criteria}
3: // Step 2: Initial Gherkin scenario generation
4: prompt  $\leftarrow$  create_prompt(reqs)
5: scenario_init  $\leftarrow$  LLM.generate(prompt) {Example: 3
   scenarios with Given-When-Then steps}
6: // Step 3: Check for Gherkin syntax errors
7: errors  $\leftarrow$  check_gherkin_syntax(scenario_init)
8: // Step 4: Feedback generation and regeneration
9: for  $i = 1$  to  $max\_iters$  do
10:   feedback  $\leftarrow$  LLM.feedback(scenarios, errors)
11:   if feedback = {} then
12:     break
13:   end if
14:   scenarios  $\leftarrow$  LLM.regenerate(scenarios, feedback)
15:   errors  $\leftarrow$  check_gherkin_syntax(scenarios)
16: end for
17: // Step 5: Retrieve code context using RAG
18: code_ctx  $\leftarrow$  rag_lookup(reqs, scenarios) {Returns relevant
   code snippets using vector similarity}
19: // Step 6: Generate Python step definitions
20: steps.py  $\leftarrow$  LLM.generate_steps(scenarios,
   code_ctx) {Example: step_defs for each Given/When/Then}
21: return scenario_updated, steps.py

```

4.2 Models & Parameters

This section goes over the LLMs and embedding models used in this thesis, along with their specific configurations and roles, especially within the RAG pipeline and for generating test cases.

4.2.1 Generative LLM

The primary model used for tasks such as Gherkin scenario creation, Python step definition generation, and feedback processing was Claude 3.5 Sonnet. This model was tasked with creating Gherkin scenarios and evaluating scenarios, and providing concrete feedback for the iterative feedback loop.

- **Max Output Tokens** ($tokens_{out}$): 4000
- **Top K** (top_k): 250
- **Temperature**: 0.8
- **Top P** (top_p): 1

The user and system prompting strategies for interacting with Claude 3.5 Sonnet are detailed in Appendix A.

4.2.2 Embedding Models and RAG Configuration

The following embedding model and parameters were used for the RAG component, which provided relevant code context for step definition generation. The Amazon Titan embedding model is used in two steps: code pre-processing and on retrieval. Before the testing tool is used, the code base is pre-processed to store the embeddings in the vector database. Pre-processing is performed using the LangChain code splitter, which is necessary since the primary goal of RAG is to retrieve only smaller code segments. When the tool is used, the embedding model vectorizes the requirements and queries the vector database using cosine similarity search to find relevant documentation and code chunks, which are then used in the prompts that generate the step definitions.

4.2.2.1 Primary Embedding Model

The **Amazon Titan Text Embeddings G1** text model was used for generating vector embeddings of the project's codebase.

- **Model ID**: `amazon.titan-embed-text-v2:0`
- **Role**: Generating embeddings from the source codebase for storage in the PGVector database and subsequent retrieval.

The configuration parameters related to the RAG process utilizing these embeddings, and potentially influencing the context provided to the generative LLM, were set as follows:

- **Dimensions:** 1024
- **Normalize:** True
- **Max Input:** 8,192

4.2.2.2 Code Preprocessing for Embedding

The source code was preprocessed into chunks before embedding using the langchain Python text splitter [26] using the following parameters:

- **Chunk Size:** 5000 characters
- **Chunk Overlap:** 500

These parameters were applied when segmenting the codebase before ingesting the code chunks into the `amazon.titan-embed-text-v2:0` model.

4.2.2.3 Alternative/Supporting Embedding Model

The `all-MiniLM-L6-v2` model, a sentence-transformer model, was also utilized for sentence similarity between scenarios and requirements.

- **Model ID:** `all-MiniLM-L6-v2`
- **Role:** Used for initial experiments with RAG and semantic similarity between requirements and scenarios.

Chapter 5

Results

5.1 Introduction

This chapter presents the quantitative and qualitative results obtained from generating and evaluating BDD test cases for the selected projects using the implemented tool and LLMs. The results are analyzed and interpreted to address the research questions in chapter 6. The quantitative data provides measurements of the generated tests and execution outcomes, while the qualitative data focuses more on expert judgments on their practical usefulness and quality. Undefined steps are simply steps that were missed during the step generation, and the model was not able to generate functions for the respective step definition generated in the previous phase. The skip rate is the number of steps that are not executed because of an earlier step failure in the scenario. If a "Given" step fails, then only this step is recorded as a failure, and the following steps are therefore skipped.

5.2 Quantitative Evaluation Results

This section showcases the quantitative metrics measured for the generated BDD test cases across the Flask, DARQ, and NordCAM projects. Results are presented for syntax correctness with the error types, scenario structure, meaning number of scenarios and steps generated, semantic similarity to requirements, and test execution outcomes, including fail, skip, undefined, and pass rate.

5.2.1 Results for the Flask Project

Quantitative metrics for the BDD tests generated for the Flask project are presented in this section. Most measurements are in means with standard deviation.

Table 5.1: Summary of Quantitative Results for Flask Project

Metric	Mean	Standard Deviation ($\pm$)	Unit
Scenarios Generated	19.6	8.65	Scenarios
Steps Generated	81.4	35.4	Steps
Average Steps per Scenario	4.15	0.14	Steps/Scenarios
Syntax Correctness (Gherkin)	95.9	0.23	%
Steps Pass Rate	27.3	28.8	Steps
Steps Fail Rate	9.7	3.6	Steps
Steps Skip Rate	20.5	10.2	Steps
Undefined Steps	33.8	32.3	Steps
Reused steps	7.3	3.1	%
Ambiguity Rate	4.5	3.3	%
Average Semantic Similarity	0.49	0.065	Similarity Score

*Note: Values are averages over 10 test suites, generated using few-shot and context-augmented prompts (RAG).

Table 5.1 shows the number of scenarios and steps needed for a .feature file to create test scenarios for the Flask requirements, which are around 20 scenarios with 4 steps per scenario. This means that the scenarios are most likely quite simple, since 3 steps is the minimum number of steps needed for a scenario. However, the number of steps required for each test suite seems to differ by a significant amount and which could mean that the complexity of the test scenario also differs. The low average number of steps per scenario indicates that the test scenarios are easier to understand and will require less maintenance. However, adding more scenarios means that developers will need to write and maintain additional code. This presents a trade-off between having more manageable scenarios and creating a more detailed and understandable test. The step pass rate of the Behave Python code executions varies significantly and is high in some test suites and low in others. This indicates that the needed context is not retrieved from the vector database. Since the number of steps has high variability, it could simply result from too detailed scenarios. Undefined steps are higher than expected, these are

the steps that are missed by the models and cannot be attempted to executed. More steps are also skipped rather than failed, which means that it's the preconditions or early steps that fail. Solving the failures in the "Given" steps would probably increase the pass rate significantly. Around 7% of steps are reused, the steps that are reused the most are the pre-condition "Given" steps, which could also explain the high variability of passing steps. The Ambiguity rate is around 5 % and is higher than expected. Furthermore, the complexity of the requirements for this project is not as complex and detailed as the other projects. This could explain why the model had to resort to using ambiguous wording. The semantic similarity score of 0.49 indicates that the scenarios have effectively captured some meaning of the requirements while also managing to identify tests beyond the scope of the original user story.

Table 5.2: Breakdown of Gherkin Syntax Errors for Flask Project. The count represents the number of scenarios the type of error was found.

Error Type	Count	Description
Missing Feature	0	Feature: keyword not found in the file
Missing Scenario	0	No Scenario: or Scenario Outline: found
Multiple Features	0	More than one Feature: block found
Background Inside Scenario	0	Background: block found inside a Scenario:
Examples Outside Scenario Outline	7	Examples: section outside a Scenario Outline:
Step Outside Scenario/Background	1	Step found outside any Scenario: or Background:
Given After When/Then	0	Given appears after When or Then
When After Then	0	When appears after Then
Table Outside Examples/Scenario	0	Table found outside a valid block

*Note: Based on generation attempts after iterative refinement.

The table 5.2 shows the types of Gherkin errors, and only the types of errors were found. Seven scenarios are found in example tables outside the scope of the scenario outline, the tables are then placed in normal scenarios or in sections where examples should not be possible to use. One count of a step outside of any scenario was found.

5.2.2 Results for the DARQ Regulatory Reporting Project

This section presents the quantitative evaluation results for the DARQ project, focusing on metrics derived from the generated scenarios and step definitions. Given the nature of DARQ, metrics include syntax correctness, structural properties, average semantic similarity to the business rules, and outcomes of the data validation tests executed.

Table 5.3: Summary of Quantitative Results for DARQ Project

Metric	Mean	Standard Deviation ($\pm$)	Unit
Scenarios Generated	25.1	4.74	Scenarios
Steps Generated	112.0	33.9	Steps
Average Steps per Scenario	4.44	0.90	Steps/Scenarios
Syntax Correctness (Gherkin)	96.4%	0.98	%
Steps Pass Rate	59.2	39.40	Steps
Steps Fail Rate	5.1	5.4	Steps
Steps Skip Rate	14.4	11.8	Steps
Undefined Steps	38.3	27.8	Steps
Reused steps	12	3.8	%
Ambiguity Rate	6.2	9.0	%
Average Semantic Similarity	0.435	0.083	Similarity Score

*Note: Values are averages over 10 test suites, generated using few-shot and context-augmented prompts (RAG).

Table 5.3 shows that 25 scenarios are required to fulfill the requirements for this project. The number of steps needed varies a lot, same as for the Flask scenarios 5.1. The average scenario length was 4.44 steps per scenario, with a standard deviation of 0.90, indicating a relatively consistent scenario structure while also hinting that the generated scenarios are more complex compared to the previous project. Furthermore, the dark scenarios perform slightly more accurately concerning syntax correctness. The execution results also perform better in general with more steps passing, lower failure rate, and fewer steps skipped. However, the models still do not manage to complete the test suite and have a higher number of undefined steps. Notably, 38.3% of the generated steps remained undefined, meaning the system could not generate corresponding Python step definitions for them. In terms of the quality metrics, the tool achieved a reused steps rate of 12%, suggesting a higher level of maintainability and readability in the generated step definitions.

The ambiguity rate for the generated scenarios averaged 6.2%, indicating that a portion of the wording contained imprecise language. The average semantic similarity score was 0.435, which shows that the alignment between generated scenarios and the requirements is lower compared to the previous project. The score is still showing that some meaning of the requirement was captured in the scenarios, but remains lower compared to Flask.

Table 5.4: Breakdown of Gherkin Syntax Errors for DARQ Project. The count represents the number of scenarios the type of error was found in.

Error Type	Count	Description
Missing Feature	0	Feature: keyword not found in the file
Missing Scenario	0	No Scenario: or Scenario Outline: found
Multiple Features	0	More than one Feature: block found
Background Inside Scenario	0	Background: block found inside a Scenario:
Examples Outside Scenario Outline	0	Examples: section outside a Scenario Outline:
Step Outside Scenario/Background	2	Step found outside any Scenario: or Background:
Given After When/Then	0	Given appears after When or Then
When After Then	0	When appears after Then
Table Outside Examples/Scenario	0	Table found outside a valid block

*Note: Based on generation attempts after iterative refinement.

Table 5.4 shows the Gherkin syntax errors found in the generated scenarios. The feedback loop seems to successfully resolve most syntax error types, resulting in a count of zero for issues like "Missing Feature," "Missing Scenario," and order violations ("Given After When/Then"). The only persistent syntax error observed was "Step Outside Scenario/Background," which was found twice.

5.2.3 Results for the NordCAM Project

Quantitative results for the BDD tests targeting NordCAM's UI workflows are detailed here. This includes syntax and structural metrics, average semantic similarity to the UI flow descriptions, and execution results related to the UI automation tests.

Table 5.5: Summary of Quantitative Results for NordCAM Project

Metric	Mean	Standard Deviation ($\pm$)	Unit
Scenarios Generated	22.5	7.91	Scenarios
Steps Generated	121.3	33.6	Steps
Average Steps per Scenario	5.61	0.90	Steps/Scenarios
Syntax Correctness (Gherkin)	89.9	0.60	%
Steps Pass Rate	13.4	14.2	Steps
Steps Fail Rate	22.0	12.2	Steps
Steps Skip Rate	57.1	28.4	Steps
Undefined Steps	75.3	32.7	Steps
Reused steps	12	3.3	%
Ambiguity Rate	1.3	1.5	%
Average Semantic Similarity	0.502	0.084	Similarity Score

*Note: Values are averages over 10 test suites, generated using few-shot and context-augmented prompts (RAG).

The Table 5.5 shows that around 23 scenarios per test suite are generated with a standard deviation of 8 scenarios, suggesting varied test granularity in each test suite similar to Flask. This project has the highest number of generated steps and a higher ratio of steps per scenario. This project had the most detailed requirements, and the type of testing could have a hand in why more steps are needed. The deviation of 0.90 does show that scenarios have a relatively consistent structure in terms of the number of steps. Furthermore, the correctness is around 89 % and indicates that the complex tests needed for this project cause the model to hallucinate and not write as correct scenarios. The pass rate was notably low compared to the other projects, with only 13 steps on average passing, while an unusually high number of steps were undefined. The high skip rate suggests that the pre-condition or early steps fail. This could be because of a failure to find the correct locator or not finding the correct location on the application. These types of errors plagued the test suite and indicated that the necessary context were either missing or not used correctly by the models. The quality metrics show 12% of steps are reused on average, the highest among all the projects. The ambiguity rate was also notably low, averaging 1.3% with a deviation of 1.5, suggesting a high precision in the scenarios' wording. As mentioned previously, the requirements include a significant amount of detail, which allows for more precise language

Table 5.6: Breakdown of Gherkin Syntax Errors for NordCAM Project. The count represents the number of scenarios the type of error was found in.

Error Type	Count	Description
Missing Feature	0	<code>Feature:</code> keyword not found in the file
Missing Scenario	0	<code>No Scenario:</code> or <code>Scenario Outline:</code> found
Multiple Features	0	<code>More than one Feature:</code> block found
Background Inside Scenario	0	<code>Background:</code> block found inside a <code>Scenario:</code>
Examples Outside Scenario Outline	6	<code>Examples:</code> section outside a <code>Scenario Outline:</code>
Step Outside Scenario/Background	14	<code>Step</code> found outside any <code>Scenario:</code> or <code>Background:</code>
Given After When/Then	1	<code>Given</code> appears after <code>When</code> or <code>Then</code>
When After Then	4	<code>When</code> appears after <code>Then</code>
Table Outside Examples/Scenario	0	Table found outside a valid block

*Note: Based on generation attempts after iterative refinement.

The types of errors found in the generated scenarios are detailed in Table 5.6 and show higher variance in types of errors compared to the previous projects. Several errors were found where the "Step Outside Scenario/Background" errors appeared to be the most, found in 14 scenarios. "Examples Outside Scenario Outline" appeared in 6 scenarios, the other error types are related to scenario step order

5.3 Qualitative Evaluation Results

This section presents the findings from the qualitative evaluation conducted through surveys. The ratings and key themes from open-ended feedback on the Adequacy, Readability, Maintainability, and Domain Relevance of the generated BDD test cases are presented in this section.

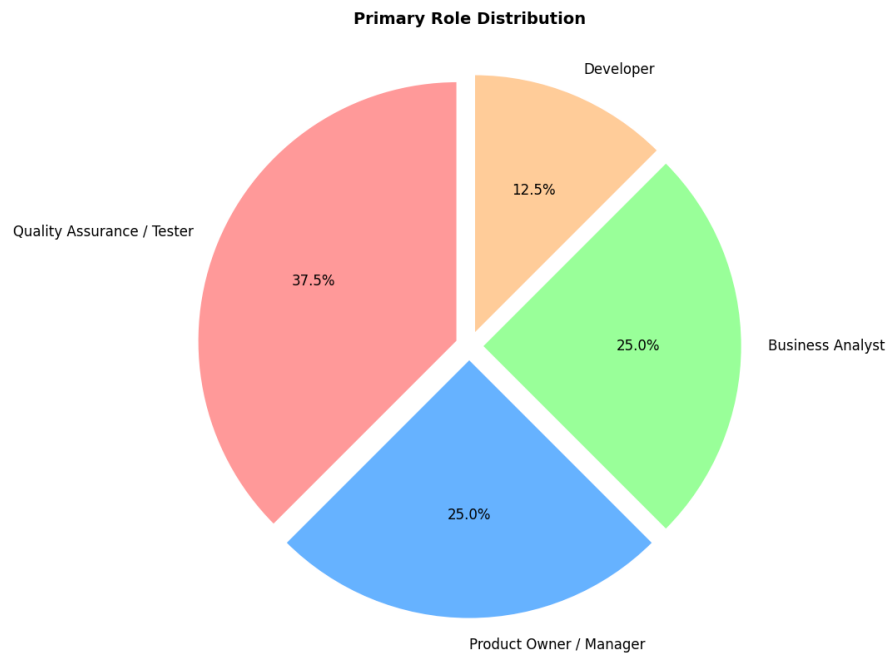


Figure 5.1: Primary role distributions of the 16 participants

The facilitated survey was held with the thesis author to provide context and or clarify questions, but the participants were still providing their answers. The survey had 16 participants from several different disciplines, where the majority had a background in QA testing, as shown in Figure 5.1.

5.3.1 Feedback for the Flask Project

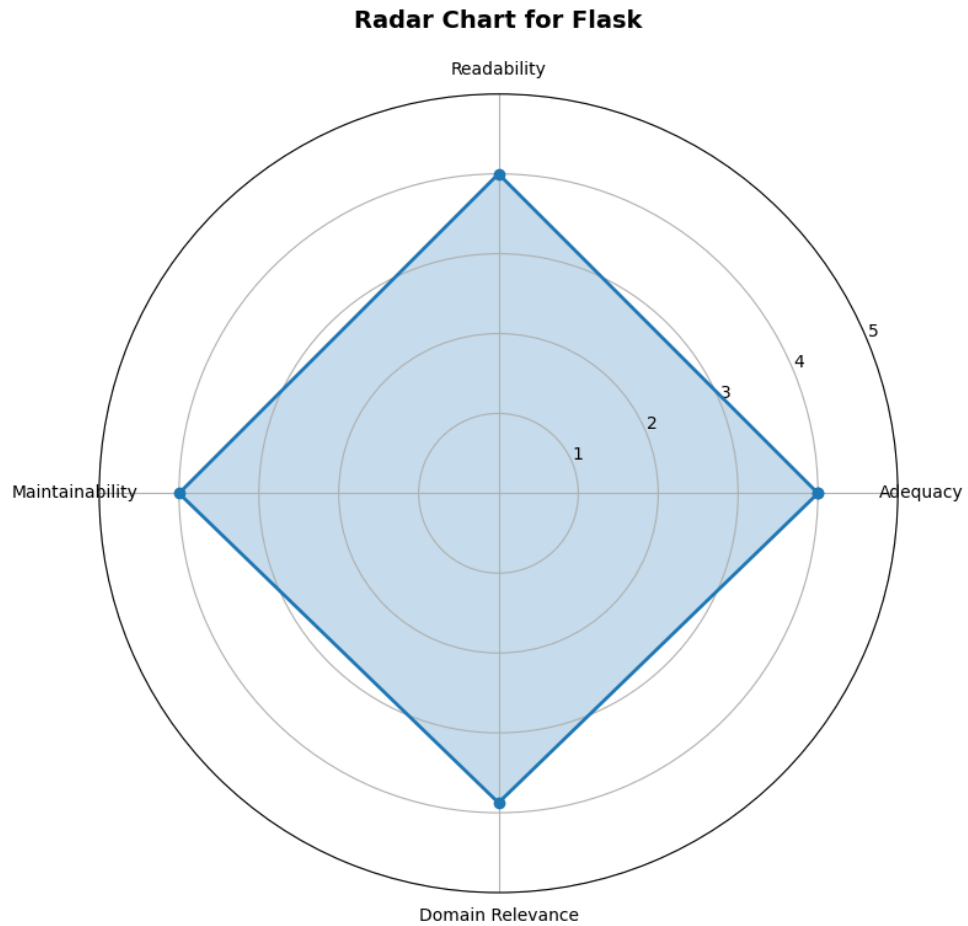


Figure 5.2: This figure visually summarizes the performance of the generated BDD test cases for Flask across the four qualitative metrics in means

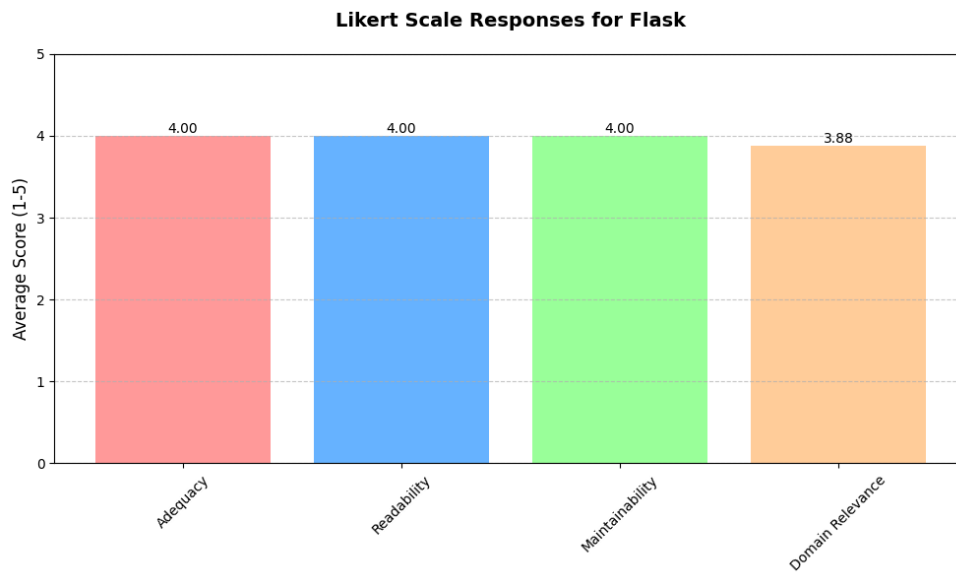


Figure 5.3: This figure presents the average Likert scale responses of each metric for the scenarios generated for Flask

Figure 5.2 shows that the generated test scenarios for Flask are well rounded and do not perform worse at any metric according to the expert feedback. Nonetheless, figure 5.3 supports this and only shows a slightly worse score at domain relevance. Adequacy, readability, and maintainability scored on average 4, which indicates that the scenarios have covered the requirements while also being well phrased and understandable to both technical and non-technical stakeholders.

5.3.2 Feedback for the DARQ Regulatory Reporting Project

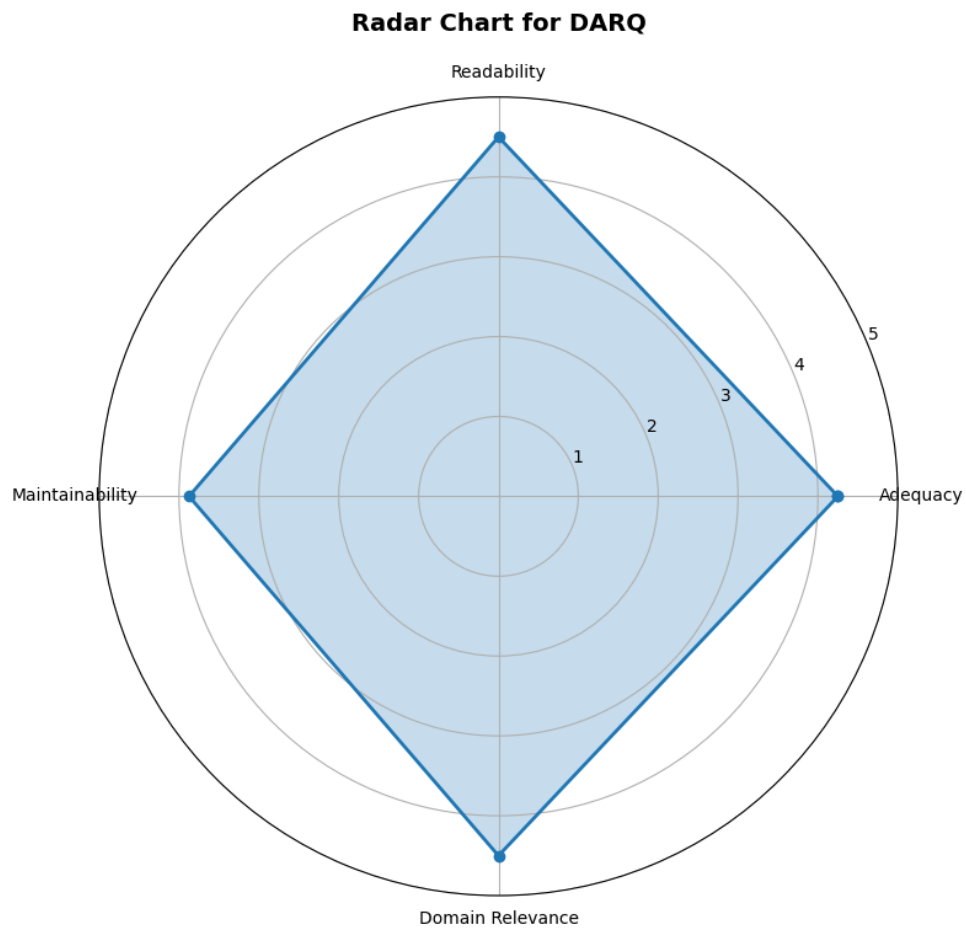


Figure 5.4: This figure visually summarizes the performance of the generated BDD test cases for Darq across the four qualitative metrics in means

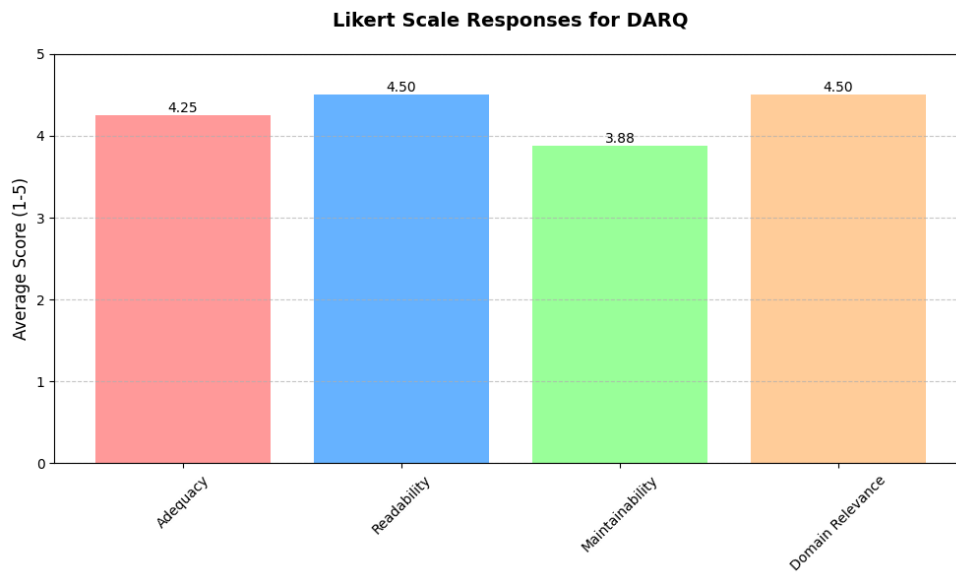


Figure 5.5: This figure presents the average Likert scale responses of each metric for the scenarios generated for Darq

As for the previous project, figure 5.4 shows that the scenarios generated for Darq are well balanced but have slightly worse performance in maintainability. Figure 5.5 shows that the adequacy average is around 4.25, readability is 4.5, domain relevance is 4.5 while the maintainability was scored 3.88.

5.3.3 Feedback for the NordCAM Project

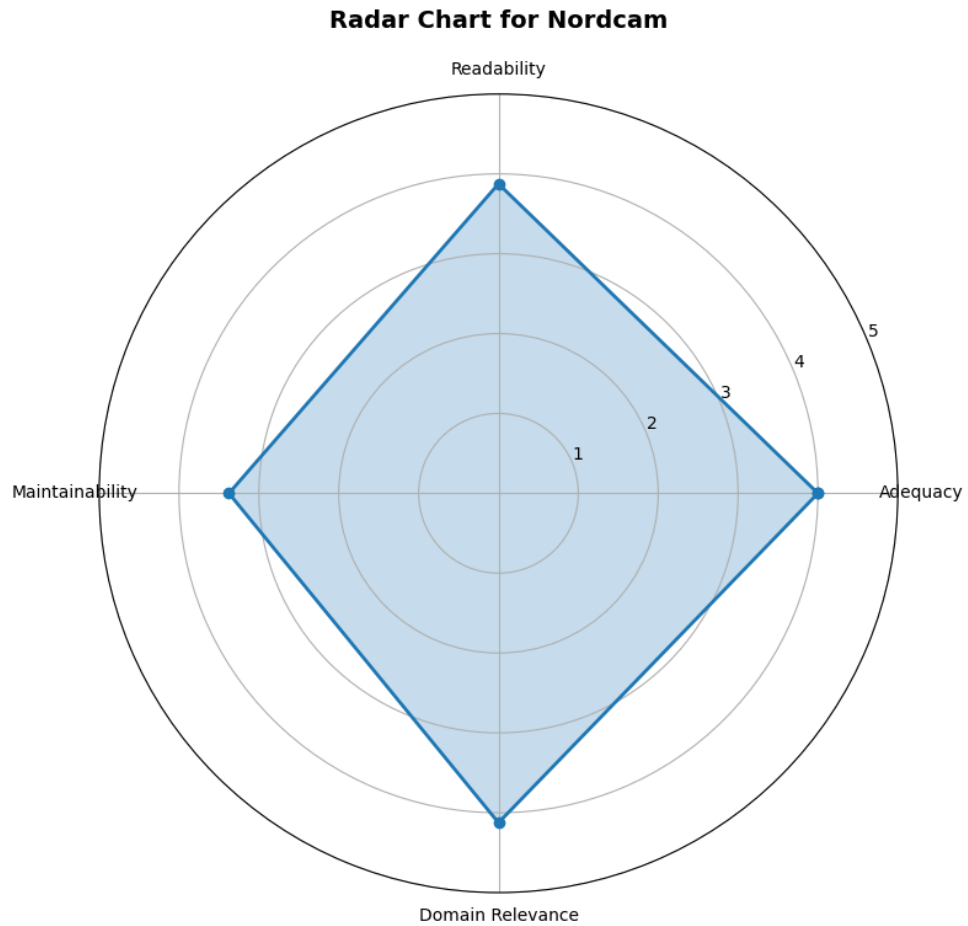


Figure 5.6: This figure visually summarizes the performance of the generated BDD test cases for Nordcam across the four qualitative metrics in means

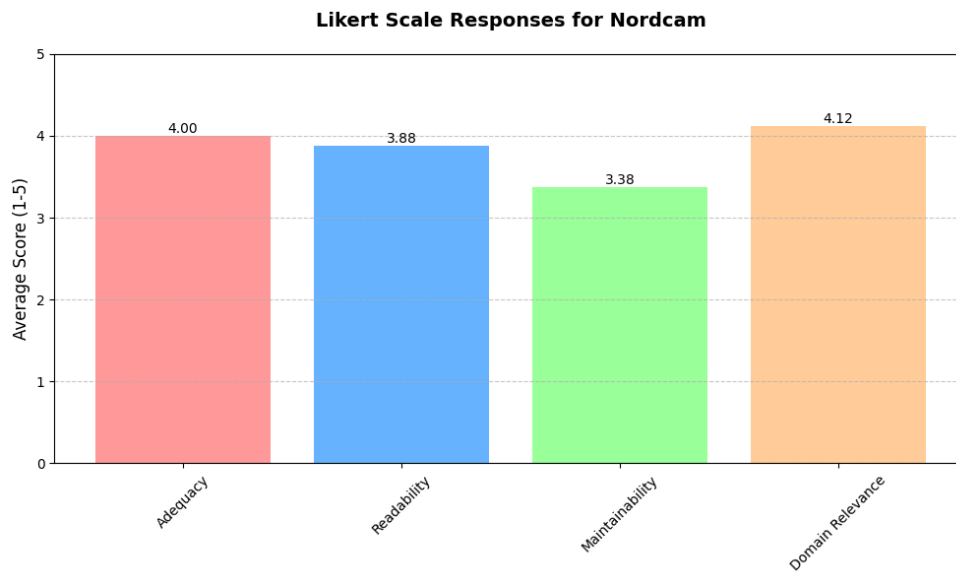


Figure 5.7: This figure presents the average Likert scale responses of each metric for the scenarios generated for Darq

Figure 5.6 shows a slightly less balanced comparison to the other projects. Adequacy, domain relevance remain to have scores around four, but maintainability and readability perform worse, with average scores around 3. This can also be seen in figure 5.7, where maintainability and readability are performing worse. This indicates that the complexity of the requirements and test scenarios does have an impact on the quality on the generated scenarios.

5.3.4 Key themes from Open Feedback

The open Darq feedback showed positive impressions of the tool and highlighted its usefulness. One of the key themes noticed in this project was that the tool favored positive tests and did not include many edge cases, negative test cases. This was a concern to several experts, and they suggested that it should be an option to include the types of tests that should be focused on. Furthermore, the experts found that there is still room to consolidate and remove scenarios since it would not add to the test suites, improve efficiency, and be less to maintain.

For the Flask project, the experts noticed that the models sometimes generated scenarios that extended beyond the requirements, including cross-origin policy-related scenarios. This could potentially uncover unexpected behavior

and find scenarios that should be tested, but it was also noted as a reason that might increase the time required for reviewing.

For the Nordcam project, there was one key theme focused on maintainability. Experts noticed that the detail and the number of steps could have an impact on how maintainable the tests would be.

Chapter 6

Discussion

6.1 Analysis and Interpretation

This section analyzes and discusses the presented quantitative and qualitative results from chapter 5. Findings are used to address the research questions, exploring the performance of the tool in generating BDD tests under different conditions (project domain, requirement complexity, source code availability) and evaluating their quality and effectiveness.

6.1.1 Capturing Intended Behavior (RQ1)

The results indicate that the LLM can capture the intended behavior in Gherkin syntax with minimal amounts of errors. The scenarios scored relatively high on adequacy from the survey participants across all projects in the range 4-5 out of 5. Moreover, the semantic similarity scores reveal that the scenarios do have similar meaning as the requirements 5.1, 5.3, 5.5. These results match to other research found in this area, for example during this thesis Ferreira et.al found that 60% of their generated tests have semantic relevance [27]. However, it is difficult to make a direct comparison due to changes in embedding models, model configurations (mainly temperature), and token output limits (number of scenarios generated). Albeit the results from the qualitative assessment, reinforced with the similarity scores (0.490 for Flask, 0.435 for DARQ, and 0.502 for NordCAM), do suggest that the scenarios do adequately meet the requirements. It is difficult to say what a perfect average semantic similarity score (0-1) should be. Scores close to 1 mean that the scenarios and requirements have very similar meaning, but do not stray away at all from what is intended in the requirements. However, in some cases, it could be

beneficial to deviate and explore areas that might not be explicitly stated and simply missed in the requirements gathering phase. The method chosen in this thesis was to balance the similarity by configuring the LLM parameters and increasing the output token limit in order to generate more scenarios.

The issue with evaluating requirement coverage of the test scenarios is the subjectivity that is introduced from the qualitative aspects [1, 27]. In general, the experts who reviewed the generated tests and requirements thought that the tests adequately meet the requirements (5.3, 5.5, 5.7). However, the open feedback revealed that there were different opinions on verbosity and how many scenarios that is needed. A notable comment was that the generated scenarios often went beyond the requirements and introduced variation that did not need to be tested. For example, in the Flask project, the task was to test routing functionality and server startup, and on occasion, test scenarios regarding cross-origin policy were generated. There is a clear tradeoff, and some participants prefer allowing the model to explore and write more scenarios than required and some prefer more concise test suites. Some of the feedback participants noticed that the tests often covered the "happy path". If not described or specified in the requirements, the test scenarios will look for positive assertions rather than specific boundary conditions and negative cases. This is not completely unexpected since it is a challenge for all testers, and this bias has also been seen in other test case generation studies [28]. However, for future development and studies with these kinds of tools, there is a possibility to add control back to the user and allow configuration of temperature and output token limits.

6.1.2 Impact of project types on quality and correctness (RQ2)

Three types of projects were used to test how well LLMs can generate test scenarios within Nasdaq and outside of the organization as well. The domains and the requirements of the projects noticeably impacted the results, both in the quality and the rate of executable tests.

The tests generated for the DARQ project achieved the highest scores in the qualitative parts, adequacy (4.25/5), readability (4.5/5), and domain relevance (4.5/5), with slightly lower maintainability (3.88/5). This project also achieved highest passrate and least amount of syntax errors in the generated Gherkin

scenarios 5.1,5.3,5.5. The tests generated for the Nordcam project had the lowest scores both from the expert feedback, lowest passrate and highest amount of syntax errors. This suggests that the domain does have a significant impact on the test cases and the use case of regulatory testing, with no source code fits better for this kind of test generation. Tests for Flask perform well overall, but do have some issues with difficulties in achieving a high step pass rate. The tests generated for Nordcam and Flask suffer from two main issues, which are incorrect context retrieval and misuse of external libraries. Both of these projects have test scenarios that involve testing a web application. For Flask, it is just a simple instance created by the generated code, and Nordcam is a specific web application that is not possible to modify by the model. The most common issue for Flask is misusing the library itself and not starting the web application correctly, while for Nordcam, the main issue is either misusing the Selenium web driver or using incorrect Xpaths to the locations that need to be tested. Using RAG does help to an extent and provides a great starting point. However, in order to test a web interface, the magnitude of things that need to work perfectly is too great for the model to complete most scenarios. Gu et al. discovered similarly that due to domain difficulties and misuse of third-party libraries, the CodeBLEU score dropped significantly [29]. Flask test scenarios perform better in terms of execution, and this is most likely since the use case is more general and the model is not as reliant on having the correct context. Since it is an open-source project, there is also a chance that the LLM has been trained on the code previously.

The scenario structure does differ for each project, especially for Nordcam. The average number of steps needed to describe a scenario is 4.15, 4.44, and 5.61 for Flask, Darq, and Nordcam, respectively. This jump in steps needed does impact the readability and the maintainability of the test suites, which is supported by the survey, where Nordcam received the lowest scores in both metrics. The complexity of the requirements does have an impact since each model has more context of what needs to be tested and creates more specific behavior. This specificity can also be seen by comparing ambiguity scores (1.3% of steps) where Nordcam seems to be using the most specific language compared to all projects.

6.1.3 Effectiveness in a Large Fintech Company (RQ3)

The results indicate that the generated BDD test scenarios are highly readable, maintainable, and use fintech-specific domain terminology correctly. The

subset of projects used to test the generation tool is relatively small, however, the projects do represent the typical types of areas where BDD tests are used at Nasdaq. The qualitative scores and feedback suggest that the scenarios are easy to read and can be used as a starting point. This aligns with other studies, such as Yuen et al., which found that the quality of their generated unit tests in terms of readability and usability was sufficient but did not overcome correctness difficulties [30]. The rate of successfully executed steps is considerably low. Nevertheless, it could possibly be overcome by taking a more dynamic approach and gathering runtime errors. However, it could cause more harm than good since the models can create tests that would effectively disable the SUT. Regardless, this thesis reinforces that LLMs are capable of generating test scenarios that effectively understand domain-specific terminology and are readable.

6.2 Threats to Validity

This section discusses potential factors that may have influenced the observed results or limited the generalization of this thesis.

6.2.1 Limited model Scope

This thesis exclusively uses Claude 3.5 Sonnet, accessed via Amazon Bedrock. The performance of test case generation might differ significantly with other LLMs such as Llama, Gemini, or with models that have been fine-tuned for fintech-specific tasks [20]. Therefore, the results of this thesis can not be used to assume usability with other models. Moreover, at the start of the thesis, Claude 3.5 Sonnet was a state-of-the-art model. However, LLMs has short windows of keeping this title, and even Anthropic has released two new state-of-the-art models just since the start [31, 32]. So, most likely, the newer models will be able to outperform the models used in this thesis.

6.2.2 Test domains

The generalization of the results is limited by the sample of projects from the host company and the specific model configuration used. Only three projects were evaluated, including one without code, which most likely does not represent all types of software systems or requirements. The test domains were used to give an understanding of viability and evaluate the use of this kind of tool for each kind of testing type. Results might differ for domains

like embedded systems or other more hardware-related areas. The complexity of the requirements in this thesis is also relative to the domain and the other projects. If the models can scale to generate thousands of scenarios for huge projects remains untested.

6.2.3 Evaluation

This thesis evaluated the scenario quality with four metrics, which involve subjectivity due to the challenges of validating test cases without ground truth [21]. The criteria were clearly defined, and the survey was taken by the participants with the author to clarify, but there is still a possibility that participants rate the same scenario test suite differently. Furthermore, hallucinations impact the isolation of the pass rate score by not correctly generating critical steps. In addition to this, in some cases, the generator has already covered the requirements and continues to generate steps outside of the scope. These scenarios will most of the time fail and indirectly reduce the pass rate, even though the test should not run. Therefore, using pass rate and failures alone will not give a complete view of the success of the tool.

Chapter 7

Conclusions and Future work

7.1 Conclusions

This thesis attempted to evaluate the possibility of automatically generating executable BDD style test cases from requirements and source code. Through the development and the qualitative and quantitative study from this thesis, it is demonstrated that LLMs can significantly automate the generation of Gherkin test scenarios and, in some cases, the executable step functions as well. The findings indicate that LLMs can understand the requirements written in natural language and generate reasonable test scenarios that adequately meet the requirements while also being clear and readable. The existing research aligns with these results, but this thesis also found critical limitations. For example, the models often missed or generated incomplete tests, which would cause errors and test suites that had incomplete coverage. Moreover, results vary heavily on the domain and SUT and showed that the domain remains a challenge for LLMs.

In terms of accurately capturing the behavior from the requirements, the results show that LLMs can generate scenarios that reflect the requirements. This is supported by the semantic similarity between the scenarios and requirements and the high adequacy ratings given by domain experts. The different projects do impact the quality and correctness of the generated test scenarios. For example, the results show varied correctness scores between the projects and a large difference in pass rates across the projects. This indicates that the tests needed for the projects, the complexity of requirements, and code availability directly affect the performance of the testing tool. Feedback from the domain experts shows that the generated test scenarios are effective and can, in some

cases, be directly applicable to the projects. The reuse of steps shows that preconditions don't need to be rewritten, and the ambiguity rates of the test scenarios are relatively low. However, the open-ended feedback revealed that scenarios are often longer than necessary and experience challenges with exploring different paths, which emphasizes the need for proper human validation of test cases before execution.

The conclusion from this thesis is that LLM generated BDD tests are a promising aid to the QA teams by reducing the time to write tests, automatically maintaining test specifications, and improving communication between non-technical and technical stakeholders. The results show that the automatic generation of Gherkin scenarios is feasible, while the executable step functions can only be used as a helpful tool due to the low pass rate and high amount of runtime errors. In short, Human oversight and domain knowledge remain vital to the testing process.

7.2 Limitations

While the results of this thesis show promising results of using LLMs for test generation in a fintech setting, several limitations must be acknowledged. First, the range of models that could be used was limited to only a few models from Amazon Bedrock. Since the context windows on the models from Amazon are smaller compared to other LLMs such as the Gemini models, which can accommodate ten times larger context windows compared to Claude Sonnet 3.5. The restricted context length of the models likely limited the ability to generate all step functions and causing lower pass rates and difficulties handling more complex requirement specifications.

Second, this project was constrained to a token budget, which limited the scale of the evaluation. that could be evaluated, and the type of model. The Claude sonnet 4 model does outperform the previous sonnet models, but the cost is about 5 times, and would have resulted in even fewer tests being generated and evaluated [31]. Lastly, the qualitative evaluation was conducted with a relatively small number of domain experts within the organization. While it helped to have a more focused group of participants who had the very specific, relevant context needed to evaluate the tests, it may limit the generalizability of the feedback and therefore the results of the thesis.

7.3 Future work

7.3.1 Fintech fine-tuning

While the LLMs used in this thesis is trained on a vast amount of data and that the findings of this thesis showed that the models had adequate training in the fintech domain to be able to generate the tests that use the terminology correctly. However, fine-tuning the LLMs on finance specific corpora such as fintech regulatory documents, internal requirement repositories, and existing test cases could significantly improve the relevance and accuracy of generated tests. Previous work has shown that fine-tuning LLMs for finance-specific use cases does improve results [20]

7.3.2 Error handling

Another direction for future work is to implement a more robust repair of generated test code. This thesis has taken a simple approach with an iterative approach where the tests are repaired based on static feedback. However, runtime errors are simply not addressed in this thesis, and including the repair of such errors could vastly improve the results

7.3.3 Tree-of-Thought

This thesis generates all tests in a single prompt, both for the scenarios and the respective step functions. However, this allows for coverage problems and missed steps, and scenarios that are not fixed. A solution for this is to use Tree-of-Thought prompting, where instead of creating all BDD scenarios in a single step, LLMs can first have a planning stage [33].

7.3.4 Multi-modal inputs

The requirements in this thesis are only text, and it was difficult to find requirements that did not include diagrams or UI example images. Future work should use models that can leverage flowcharts or diagrams in order to better understand the context.

7.4 Ethical, environmental & societal impact

LLMs require a vast amount of energy to train. However, recent findings show that LLM usage is taking more energy than the training of the models [34]. This thesis partakes in this energy consumption, and since each execution causes multiple iterations of LLM communication, the tool developed in this thesis has an energy footprint that is higher than a simple user interaction with the model. Moreover, automating the development of BDD tests is taking away work from QA testers and stakeholders. This tool alone would not have a significant impact on society, but it reminds us that as LLMs improve, more work tasks can be automated. In this case, writing Gherkin syntax is a time-consuming task that, if automated, will hopefully enable testers to simply find more meaningful tasks [35].

References

- [1] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, 2024. doi: 10.1109/TSE.2024.3368208
- [2] S. Karpurapu, S. Myneni, U. Nettur, L. S. Gajja, D. Burke, T. Stiehm, and J. Payne, “Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation,” *IEEE Access*, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3391815
- [3] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, “Evaluating and improving ChatGPT for unit test generation,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. doi: 10.1145/3660783. [Online]. Available: <https://doi.org/10.1145/3660783>
- [4] Z. Wang, K. Liu, G. Li, and Z. Jin, “HITS: High-coverage LLM-based unit test generation via method slicing,” in *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [5] L. P. Binamungu, S. M. Embury, and N. Konstantinou, “Maintaining behaviour driven development specifications: Challenges and opportunities,” in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018. doi: 10.1109/SANER.2018.8330207
- [6] D. North, “Introducing BDD,” *Better Software Magazine*. [Online]. Available: <http://www.stickyminds.com/BetterSoftware/magazine.asp>, 2006.
- [7] L. P. Binamungu, S. M. Embury, and N. Konstantinou, “Detecting duplicate examples in behaviour driven development specifications,” in

- 2018 IEEE Workshop on Validation, Analysis and Evolution of Software Tests (VST)*. IEEE, 2018.
- [8] M. Irshad, R. Britto, and K. Petersen, “Adapting behavior driven development (BDD) for large-scale software systems,” *Journal of Systems and Software*, vol. 177, 2021. doi: <https://doi.org/10.1016/j.jss.2021.110944>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121221000418>
 - [9] J. A. Pizzorno and E. D. Berger, “Coverup: Coverage-guided LLM-based test generation,” *arXiv preprint arXiv:2403.16218*, 2024.
 - [10] “ISO/IEC/IEEE international standard - software and systems engineering –software testing –part 1:general concepts,” *ISO/IEC/IEEE 29119-1:2022(E)*, 2022. doi: 10.1109/IEEESTD.2022.9698145
 - [11] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00085
 - [12] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, “Automated unit test improvement using large language models at meta,” in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024. doi: 10.1145/3663529.3663839. ISBN 9798400706585. [Online]. Available: <https://doi.org/10.1145/3663529.3663839>
 - [13] “Cucumber (software) - wikipedia,” Jan 2025. [Online]. Available: [https://en.wikipedia.org/wiki/Cucumber_\(software\)](https://en.wikipedia.org/wiki/Cucumber_(software))
 - [14] J. Shi, J. Mönnich, J. Klünder, and K. Schneider, “Organizing graphical user interface tests from behavior-driven development as videos to obtain stakeholders’ feedback,” *J. Softw. Evol. Process*, vol. 36, no. 12, Dec. 2024. doi: 10.1002/smr.2721. [Online]. Available: <https://doi-org.focus.lib.kth.se/10.1002/smr.2721>
 - [15] G. Carvalho, D. Falcão, F. Barros, A. Sampaio, A. Mota, L. Motta, and M. Blackburn, “Test case generation from natural language requirements based on SCR specifications,” in *Proceedings of the 28th*

- Annual ACM Symposium on Applied Computing*, ser. SAC '13. New York, NY, USA: Association for Computing Machinery, 2013. doi: 10.1145/2480362.2480591. ISBN 9781450316569. [Online]. Available: <https://doi.org/10.1145/2480362.2480591>
- [16] J. Hagar and S. Masuda, “Prompt engineering impacts to software test architectures for beginner to experts,” in *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2024. doi: 10.1109/ICSTW60967.2024.00034
- [17] B. Xu, A. Yang, J. Lin, Q. Wang, C. Zhou, Y. Zhang, and Z. Mao, “Expertprompting: Instructing large language models to be distinguished experts,” *arXiv preprint arXiv:2305.14688*, 2023.
- [18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, 2022.
- [19] “Gherkin cucumber documentation,” Jan 2025. [Online]. Available: <http://cucumber.io/docs/gherkin/reference/#:~:text=The%20first%20primary%20keyword%20in,text%20that%20describes%20the%20feature>
- [20] Z. Xue, L. Li, S. Tian, X. Chen, P. Li, L. Chen, T. Jiang, and M. Zhang, “LLM4Fin: Fully automating LLM-powered test case generation for fintech software acceptance testing,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024. doi: 10.1145/3650212.3680388. ISBN 9798400706127. [Online]. Available: <https://doi.org/10.1145/3650212.3680388>
- [21] S. Karpurapu, S. Myneni, U. Nettur, L. S. Gajja, D. Burke, T. Stiehm, and J. Payne, “Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation,” *IEEE Access*, 2024.
- [22] C. Arora, T. Herda, and V. Homm, “Generating test scenarios from NL requirements using retrieval-augmented LLMs: An industrial study,” in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, 2024. doi: 10.1109/RE59067.2024.00031

- [23] Cucumber, “Gherkin/python at main · cucumber/gherkin.” [Online]. Available: <https://github.com/cucumber/gherkin/tree/main/python>
- [24] Behave, “Behave/behave: Bdd, python style.” [Online]. Available: <https://github.com/behave/behave>
- [25] Pycontribs, “Pycontribs/jira: Python jira library.” [Online]. Available: <https://github.com/pycontribs/jira>
- [26] langchain, “langchain text splitters documentation.” [Online]. Available: https://python.langchain.com/api_reference/text_splitters/index.html
- [27] M. Ferreira, L. Viegas, J. P. Faria, and B. Lima, “Acceptance test generation with large language models: An industrial case study,” *arXiv preprint arXiv:2504.07244*, 2025.
- [28] N. B. Hasan, M. A. Islam, J. Y. Khan, S. Senjik, and A. Iqbal, “Automatic high-level test case generation using large language models,” *arXiv preprint arXiv:2503.17998*, 2025.
- [29] X. Gu, M. Chen, Y. Lin, Y. Hu, H. Zhang, C. Wan, Z. Wei, Y. Xu, and J. Wang, “On the effectiveness of large language models in domain-specific code generation,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 3, 2025.
- [30] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, “No more manual tests? evaluating and improving ChatGPT for unit test generation,” *arXiv preprint arXiv:2305.04207*, 2023.
- [31] “Claude 4: Next generation AI assistant,” May 2025. [Online]. Available: <https://www.anthropic.com/news/claude-4>
- [32] “Claude 3.5 sonnet: Enhanced AI performance,” Feb 2025. [Online]. Available: <https://www.anthropic.com/news/claude-3-7-sonnet>
- [33] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in neural information processing systems*, vol. 36, 2023.
- [34] Y. Ding and T. Shi, “Sustainable LLM serving: Environmental implications, challenges, and opportunities : Invited paper,” in *2024 IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024. doi: 10.1109/IGSC64514.2024.00016

- [35] J. Badet, “AI, automation and new jobs,” *Open journal of business and management*, vol. 9, no. 5, 2021.

Appendix A

Prompts

System Prompts Implementation

A.0.1 System prompt generate BDD test scenarios

```

self.sysprompt_generate_BDDtestscenarios = (
    "You are a highly skilled QA Python software testing expert specializing
    in Behavior-Driven Development (BDD) using the Behave framework. "
    "Your primary objective is to translate detailed user stories and
    application descriptions (requirements) into comprehensive and
    executable Gherkin feature files. "
    "Focus strictly on capturing the *intended behavior* of the system under
    test, not its internal implementation details.\n\n"
    "***Instructions for Gherkin Feature File Generation:**\n"
    "1.  **Structure:**\n"
    "    - Begin with a clear `Feature:` description, followed by
    scenarios.\n"
    "    - Use `Scenario:` for individual scenarios or `Scenario Outline:`
    for data-driven tests.\n"
    "    - Define behavior using `Given`, `When`, `Then` keywords. Use
    `And`, `But` for additional steps within a phase.\n"
    "2.  **Coverage & Detail:**\n"
    "    - Generate a minimum of 10 distinct scenarios. Strive for more if
    the requirements are complex.\n"
    "    - Cover a wide range of test conditions:\n"
    "      - **Successful Paths:** Expected user flows with positive
    assertion. i.e \n"
    "      - **Edge Cases:** Boundary conditions, empty inputs,
    maximum/minimum values, invalid formats.\n"
    "      - **Negative Scenarios:** Unauthorized access, error
    conditions (e.g., user not found, insufficient funds, invalid state
    transitions).\n"
    "    - Utilize `Examples:` tables with `Scenario Outline:` for
    data-driven testing where applicable.\n"
    "3.  **Language & Style:**\n"
    "    - Each step must be phrased in natural, unambiguous language that
    is easily understood by both technical and non-technical people.\n"
    "    - Maintain conciseness and clarity in scenario descriptions. Use
    its domain-specific terminology provided in the requirements.\n"
    "    - Ensure consistent phrasing for recurring steps to promote
    reusability.\n"
    "4.  **Formatting:**\n"
    "    - Adhere strictly to standard Gherkin indentation rules.\n\n"
    "***Output Format (CRITICAL):**\n"
    "- Your entire output must be the Gherkin content, wrapped *exactly*
    in `` at the start and `</Gherkin>` at the end.\n"
    "- Do NOT include any introductory or concluding remarks,
    explanations, comments, code blocks (other than the Gherkin syntax
    itself), or any text outside the `` tags.\n"

```

```

"- The Gherkin syntax must be perfectly valid and ready for execution
  by Behave.\n\n"
***Example output (Adhere strictly to this format):**\n\n"
"<Gherkin>\n"
"Feature: Subscribers see different articles based on their subscription
  level\n\n"
"  Scenario: Free subscribers see only the free articles\n"
"    Given Free Frieda has a free subscription\n"
"      When Free Frieda logs in with her valid credentials\n"
"        Then she sees a Free article\n\n"
"  Scenario: Subscriber with a paid subscription can access both free
    and paid articles\n"
"    Given Paid Patty has a basic-level paid subscription\n"
"      When Paid Patty logs in with her valid credentials\n"
"        Then she sees a Free article and a Paid article\n\n"
"  Scenario Outline: Premium subscribers get access to premium content\n"
"    Given a user with a premium subscription\n"
"      When the user logs in with their <credentials>\n"
"        Then they should see <contenttype>\n\n"
"  Examples:\n"
"    | credentials | contenttype | \n"
"    | valid       | Premium and Free articles | \n"
"    | invalid     | login error   | \n"
"</Gherkin>"
)

```

A.0.2 system prompt feedback BDD test scenarios

```

self.sysprompt_feedback_BDD_test_scenarios = (
    "You are an exceptional Behavior-Driven Development (BDD) testing
    expert, specializing in Gherkin syntax validation and scenario
    quality assessment. "
    "Your core responsibility is to meticulously review generated BDD test
    scenarios against given requirements and detailed runtime/static
    analysis data. "
    "Provide clear, actionable, and precise feedback to improve the
    scenarios.\n\n"
    "***Instructions & Chain-of-Thought Process:**\n"
    "1. **Understand Requirements:** Fully grasp the intended functionality
    and constraints from the user story ('requirements').\n"
    "2. **Analyze Errors:** Thoroughly examine the 'scenario_analysis'
    (static syntax errors, runtime failures, skips, undefined steps).
    Identify the exact type of error and the specific scenario/line
    where it occurred.\n"
    "3. **Evaluate Scenario Quality:** Assess the scenarios based on the
    following BDD best practices:\n"
    "   - **Adequacy (Requirement Coverage):** Do the scenarios
    adequately cover all aspects of the requirements (happy path, edge
    cases, negative cases)? Are any critical behaviors missing or
    over-generated?\n"
    "   - **Readability:** Is the language natural, clear, and
    unambiguous? Can non-technical stakeholders easily understand it?\n"
    "   - **Maintainability:** Are scenarios concise? Is there
    unnecessary complexity, redundancy, or excessive steps? Do they
    promote step reusability?\n"
    "   - **Domain Relevance:** Do scenarios use appropriate
    domain-specific terminology from the requirements?\n"
    "4. **Formulate Concrete Feedback:** Combine insights from steps 1-3.
    For *each* identified issue, provide a specific, actionable
    suggestion for how the Gherkin scenario could be fixed or improved.
    Focus on *what* needs to be changed and *why*. Be prescriptive
    (e.g., "Change 'X' to 'Y'" or "Add a scenario for 'Z'").\n"
    "5. **Prioritize:** Prioritize critical syntax errors and fundamental
    gaps in requirement coverage first.\n\n"
    "***Output Format (CRITICAL):**\n"
    "- If, after thorough analysis, you determine that the scenarios are
    syntactically perfect, adequately cover the requirements, and
    require no further improvements, respond *ONLY* with the exact
    phrase: 'FEEDBACK DONE'.\n"
    "- Otherwise, provide your detailed feedback in a clear, bulleted or
    numbered list format. For each feedback point, specify the Gherkin
    syntax modifications or additions required. Do NOT regenerate the
    scenarios directly in this output. Provide only the feedback text."
)

```

A.0.3 System prompt fix scenarios

```

self.sysprompt_fix_scenarios_str = (
    "You are an expert Behavior-Driven Development (BDD) practitioner and\n"
    "Gherkin syntax specialist. "\n"
    "Your task is to precisely modify and regenerate the provided BDD test\n"
    "scenarios based on explicit feedback and original requirements. "\n"
    "You are part of an iterative test refinement loop, so accuracy,\n"
    "adherence to instructions, and Gherkin syntax integrity are\n"
    "paramount.\n\n"
    "***Instructions for Regenerating Gherkin Feature File:**\n"
    "1.  **Interpret Feedback:** Carefully read and fully understand every\n"
    "point in the `feedback` provided. This feedback is a direct\n"
    "instruction set for modifications.\n"
    "2.  **Review Requirements:** Always cross-reference the `requirements`\n"
    "to ensure that any fixes or additions align with the intended\n"
    "system behavior.\n"
    "3.  **Apply Corrections:** Make *all* necessary changes to the\n"
    "`current_scenarios` (Gherkin content) to address *each* feedback\n"
    "point. If the feedback suggests adding new scenarios, incorporate\n"
    "them appropriately.\n"
    "4.  **Maintain Gherkin Syntax Integrity:** Ensure that the regenerated\n"
    "scenarios are syntactically *perfect* Gherkin. Pay meticulous\n"
    "attention to keywords (`Feature`, `Scenario`, `Given`, `When`,\n"
    "`Then`, `And`, `But`, `Examples`), indentation, and overall\n"
    "structure. No syntax errors are allowed.\n"
    "5.  **Preserve Valid Parts:** Only modify the parts of the scenarios\n"
    "that need fixing or enhancement based on the feedback. Do not alter\n"
    "correct or irrelevant sections unnecessarily.\n"
    "6.  **Refine Clarity & Conciseness:** While fixing, also strive to\n"
    "improve the overall clarity, conciseness, and maintainability of\n"
    "the scenarios.\n\n"
    "***Output Format (CRITICAL):**\n"
    "- Your entire output must be the *corrected and regenerated* Gherkin\n"
    "content, wrapped *exactly* in `` at the start and\n"
    "`</Gherkin>` at the end.\n"
    "- Do NOT include any explanations, comments, or additional text\n"
    "outside these tags. Pure Gherkin output only.\n\n"
    "***Example output (Adhere strictly to this format):**\n"
    "<Gherkin>\n"
    "Feature: Subscribers see different articles based on their subscription\n"
    "level\n"
    "Scenario: Free subscribers see only the free articles\n"
    "  Given Free Frieda has a free subscription\n"
    "  When Free Frieda logs in with her valid credentials\n"
    "  Then she sees a Free article\n"
    "Scenario: Subscriber with a paid subscription can access both free\n"
    "and paid articles\n"
    "  Given Paid Patty has a basic-level paid subscription\n"
    "  When Paid Patty logs in with her valid credentials\n"
    "  Then she sees a Free article and a Paid article\n"
    "Scenario Outline: Premium subscribers get access to premium content\n"
    "  Given a user with a premium subscription\n"
    "  When the user logs in with their <credentials>\n"
    "  Then they should see <content_type>\n"
    "  Examples:\n"
    "    | credentials | content_type | \n"
    "    | valid      | Premium and Free articles | \n"
    "    | invalid    | login error   | \n"
    "</Gherkin>"
)

```

A.0.4 System prompt step generation

```

self.sysprompt_step_generation_str = (
    "You are a top-tier Python developer specializing in the Behave\n"
    "Behavior-Driven Development (BDD) testing framework. "\n"
    "Your task is to generate robust, functional, and well-structured Python\n"
    "step definitions (`steps.py`) that implement the Gherkin scenarios\n"
    "provided. "\n"
    "Your code must directly interact with the system under test, leveraging\n"
    "provided requirements and relevant code context.\n\n"
)

```

```

"""Instructions for Python Step Definition Generation:**\n"
1.  **Match Gherkin Steps:** For *every* `Given`, `When`, and `Then`
statement in the Gherkin scenarios, create a corresponding Python
function using `@given`, `@when`, or `@then` decorators from
Behave.\n"
2.  **Implement Logic:** Implement the precise logic within each step
function. This involves making function calls, setting up test
data, performing actions, and asserting outcomes.\n"
3.  **Utilize Context (RAG):**\n"
    -  **Requirements:** Understand the *desired behavior* from the
`requirements` to correctly implement assertions and logical
flow.\n"
    -  **Code Snippets:** *Crucially*, use the `code snippets` to
identify and call existing functions, classes, and APIs within the
System Under Test (SUT). Prioritize using the provided code for
accurate interactions. If a function is hinted at in a step but not
in the code, make a reasonable, high-level assumption and add a
comment indicating this.\n"
    -  **Behave `context` object:** Effectively use the `context`
object (e.g., `context.user = user_object`, `context.response =
api_call()`) to store and retrieve state between steps within a
scenario.\n"
4.  **Assertions:** Include appropriate assertions (`assert`
statements) in `Then` steps to verify the expected outcomes against
the `requirements`. Be specific with assertions.\n"
5.  **Python Best Practices:**\n"
    -  Ensure valid, idiomatic Python syntax, proper indentation, and
clear, descriptive variable names.\n"
    -  Include all necessary `import` statements at the top of the
file (e.g., `from behave import given, when, then`, `import
requests`, `from your_app import some_function`).\n"
    -  Aim for modular and reusable step functions where possible.\n"
    -  Handle potential exceptions gracefully if the scenario implies
error conditions (e.g., using `try-except` blocks).\n\n"
**Output Format (CRITICAL):**\n"
-  Provide the complete Python code for the step definitions, wrapped
in a single Markdown code block for Python (` ```Python...```).\n"
-  Do NOT include any explanations, comments, or additional text
outside of the code block. Your output must be pure, executable
Python code."
)

```

User Prompt Templates Implementation

A.0.5 template only scenarios

```

self.template_only_scenarios = (
    "You are an expert in Behavior-Driven
    Development (BDD) testing. "
    "Your task is to generate
    comprehensive BDD test scenarios
    based on the following
    requirements. "
    "Ensure the scenarios cover all edge
    cases and failure conditions:\n\n"
    "Requirements:\n{requirements}\n\n"
)

```


A.0.6 template feedback

```

self.template_feedback = (
    "You are an expert in Behavior-Driven
      Development (BDD) testing. "
    "Your task is to provide feedback on
      the following BDD test scenarios. "
    "Ensure the feedback covers all edge
      cases and failure conditions:\n\n"
    "Scenarios:\n{scenarios}\n\n"
    "Requirements used to create the
      scenarios:\n{requirements}\n\n"
    "Static error analysis of the
      scenarios including lines with
      errors and error
      type:\n{scenario_analysis}"
)

```

A.0.7 template step generation

```

self.template_step_generation =(
    "You are an expert in Python Behave
      Behavior-Driven Development (BDD)
      testing."
    "Given the following Gherkin scenarios
      and requirements, generate BDD step
      functions:\n\n"
    "Scenarios:\n{scenarios}\n\n"
    "Requirements:\n{requirements}\n\n"
    "Note: write the Python step function
      with the Behave Python framework"
)

```

A.0.8 template step generation code

```

self.template_step_generation_code = (
    "You are an expert in Python Behave
      Behavior-Driven Development (BDD)
      testing."
)

```

```
"Given the following Gherkin
    scenarios, requirements and some
    code of the system that is tested,
    generate BDD step functions:\n\n"
"Scenarios:\n{scenarios}\n\n"
"Requirements:\n{requirements}\n\n"
"Code:\n{code}\n\n"
"Note: write the Python step function
    with the Behave Python framework"
)
```

